VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Database Management Systems (CO3021)

Assignment 2

DATABASE MANAGEMENT SYSTEMS (CO3021)

Instructor(s): Phan Trong Nhan, *CSE - HCMUT*

Students: Pham Quang Tien Thanh *(Class CC01, **Leader**)*
Nguyen Huu Hao - 2352296 *(Class CC01)*
Le Dung - 2252131 *(Class CC01)*
Léo Floissac - 2560005 *(Class CC01)*
Youri Renoud-Grappin - 2560010 *(Class CC01)*
Arthur Bardot - 2460080 *(Class CC01)*

HO CHI MINH CITY, OCTOBER 2025

Contents

List of Figures

List of Tables

Member list & Workload

No.	Fullname	Student ID	Work	% done
1	Pham Quang Tien Thanh	2353103	- Introduction, Database Design, Deployment, SQL Server Indexing. - Query Processing for SQL Server and MongoDB - SQL Concurrency	100%
2	Nguyen Huu Hao	2352296	- MongoDB Indexing and Concurrency Control	100%
3	Le Dung	2252131	-	0%
4	Léo Floissac	2560005	- MongoDB and SQL Server Transaction - Python script	100%
5	Youri Renoud-Grappin	2560010	- MongoDB and SQL Server Transaction - Python script	100%
6	Arthur Bardot	2460080	- SQL Server Query Processing	100%

Table 1: Member list & workload

1 Introduction

1.1 Background and Motivation

In modern medical informatics and diagnostic imaging, clinical systems generate a massive volume of imaging and associated data. This includes patient information, studies, series, and—most critically—a vast number of individual image files. Storing this information in a relational database, such as the `LumbarMRI Database` used in this project, is standard practice for Picture Archiving and Communication Systems (PACS) and research repositories.

However, simply storing the data is not enough. As the database grows, particularly tables like `IMAGE` (with over 48,000 records) and `SERIES` (with thousands of records), the performance of queries used for reporting, analytics, or application functionality can degrade significantly. Simple tasks, such as finding all 'Sagittal' images for a specific patient or joining radiologist notes (`RADIOLOGISTS_DATA`) with their corresponding image files, can become inefficient and time-consuming. This slowdown is often caused by the Database Management System (DBMS) being forced to scan entire large tables or perform costly joins to find small subsets of data.

This project addresses this critical performance problem by analyzing a populated SQL Server database. The primary goal is to demonstrate how a DBMS processes queries and to implement and evaluate one of the most powerful tools for query optimization: **database indexing**. This will unlock the database's true potential, allowing for rapid retrieval and powerful data analysis, ultimately supporting better clinical research and application performance.

1.2 Project Objectives

The primary objectives of this project are:

- To deploy and analyze the schema and data relationships of the `LumbarMRI Database` sample database.
- To identify performance bottlenecks by executing realistic analytical queries on the database in its default, unoptimized state.
- To demonstrate and explain how the SQL Server query optimizer processes inefficient queries using tools like the **Actual Execution Plan** (e.g., `Table Scan` or `Clustered Index Scan`).
- To design and implement appropriate non-clustered indexes to resolve these specific performance bottlenecks.
- To prove and quantify the effectiveness of these indexes by comparing the "before" and "after" Execution Plans, showing the shift to more efficient operators like `Index Seek` and the reduction in query cost.

1.3 Methodology

This project will be conducted using a combination of theoretical research and practical application:

- **Theoretical Research:** Reviewing official Microsoft documentation and academic resources on relational database architecture, query optimization, T-SQL, and advanced indexing strategies (e.g., SARGable queries, Covering Indexes).
- **Practical Implementation:** Utilizing SQL Server Management Studio (SSMS) as the primary environment. This includes executing T-SQL scripts to deploy the `LumbarMRI Database` and then implementing custom `CREATE INDEX` scripts.
- **Testing and Validation:** Executing a series of T-SQL queries and capturing their **Actual Execution Plans**. The core of the analysis will involve a comparative study of these plans to validate the performance impact of the indexes.

2 Database Design: Analysis of the LumbarMRI Schema

The `LumbarMRI Database` provides a schema that simulates a real-world medical imaging repository. Unlike designing a database from unstructured data, this project analyzes a pre-defined schema to understand its structure, relationships, and inherent performance characteristics.

The database is structured around five core entities that logically represent the hierarchy of medical imaging data, from the patient down to the individual image file: `PATIENT`, `STUDY`, `SERIES`, `IMAGE`, and `RADIOLOGISTS_DATA`.

2.1 E-R Model and Key Relationships

The conceptual design establishes the core relationships that drive the data logic. The queries in this project focus on the flow from a patient, through their studies, to the specific images and diagnostic notes.

- **Entities (Tables):**
  - **PATIENT:** The individual undergoing the medical procedure.
  - **STUDY:** The specific MRI study event (e.g., "L-SPINE_LSS"), linked to a `PATIENT`.
  - **SERIES:** A collection of images within a `STUDY`, typically defined by a specific imaging protocol (e.g., "T2_TSE_SAG").
  - **IMAGE:** The individual image files (.ima) that compose a `SERIES`.
  - **RADIOLOGISTS_DATA:** The clinical notes or diagnosis associated with a `PATIENT`.
- **Key Relationships (for this Analysis):**
  - **Patient-has-Study (One-to-Many):** One `PATIENT` can have multiple `STUDY` records $(1 : N)$.
  - **Study-contains-Series (One-to-Many):** One `STUDY` is composed of one or more `SERIES` $(1 : N)$.
  - **Series-contains-Image (One-to-Many):** One `SERIES` is composed of one or more `IMAGE` files $(1 : N)$.
  - **Patient-has-Note (One-to-One):** One `PATIENT` has exactly one `RADIOLOGISTS_DATA` record $(1 : 1)$.

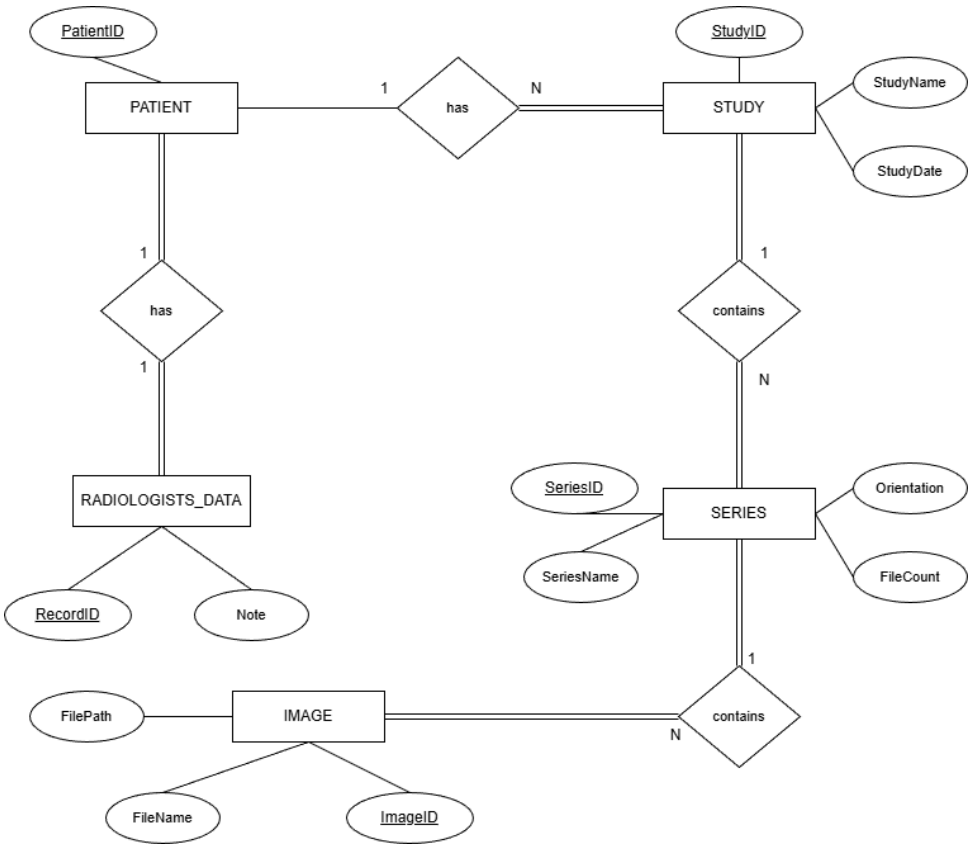


Figure 1: The (E-)ERD for the LumbarMRI System.

2.2 Logical Model and Default Indexing Strategy

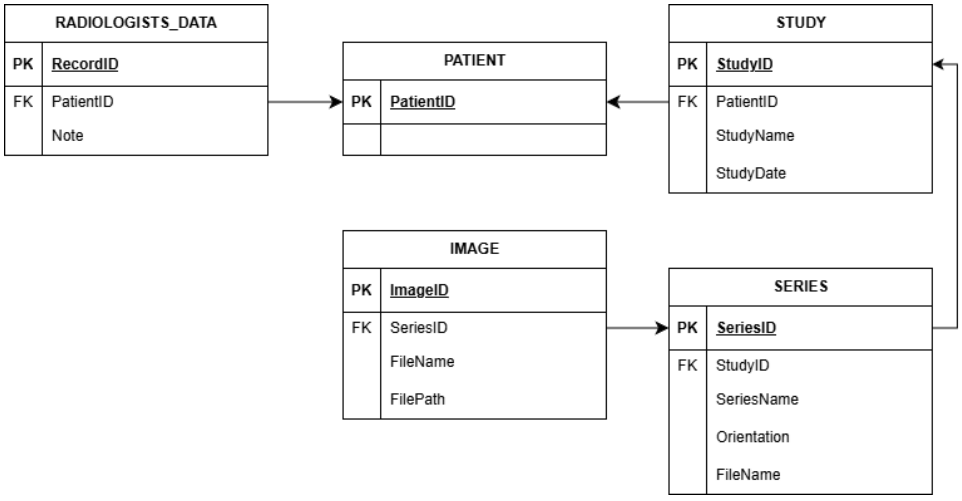


Figure 2: The Logical design of Database.

The logical model is implemented with a clear relational structure. The most critical aspect of this design for our project is its **default indexing strategy**, which is established by `PRIMARY KEY` and `FOREIGN KEY` constraints.

- **Primary Keys (Clustered Indexes):** Every table in the database has a `PRIMARY KEY` (e.g.,

PATIENT.PatientID, STUDY.StudyID, IMAGE.ImageID).

  - In Microsoft SQL Server, creating a Primary Key **automatically creates a Clustered Index** on that column.
  - This makes individual row lookups by ID (e.g., `WHERE PatientID = 100`) extremely fast, as the DBMS can "seek" directly to the data's physical location.

- **Foreign Keys (Enforcing Integrity):** The schema heavily uses `FOREIGN KEY` constraints to ensure referential integrity between tables (e.g., `STUDY.PatientID` references `PATIENT.PatientID`, or `IMAGE.SeriesID` references `SERIES.SeriesID`).
  - This design ensures that no `IMAGE` can exist without a valid `SERIES`.
  - These key relationships are what allow for the powerful, multi-table `JOIN` operations used in our analytical queries.
- **The Performance "Gap" (Motivation for this Project):**
  - Crucially, the default schema **does not** create indexes on common, non-key columns used for analytical filtering (i.e., in `WHERE` clauses).
  - **Example 1:** The `STUDY` table (558 records) has no index on `StudyDate`.
  - **Example 2:** The `SERIES` table (3,761 records) has no index on `Orientation`.
  - **Example 3:** The `IMAGE` table (48,345 records) has no index on `FileName`.
  - This "indexing gap" is the direct cause of the performance bottlenecks identified in this report. Without these indexes, any query filtering by date, orientation, or name will force the DBMS to perform a full `Table Scan` or `Clustered Index Scan`, which is highly inefficient on large tables. The following sections will demonstrate how we identify and resolve this gap.

2.3 Semantic Constraints

Beyond the core PK/FK constraints, the schema enforces data integrity through domain constraints (data types) and business rules:

- **Data Type Integrity:** Columns are defined with appropriate types (e.g., `INT`, `DATETIME`, `NVARCHAR(255)`, `NVARCHAR(MAX)`) to ensure data is stored correctly.
- **Nullability (NOT NULL):** Key columns like `PatientID` and `SeriesID` are marked as `NOT NULL`. Conversely, descriptive fields like `StudyName` or `RADIOLOGISTS_DATA.Note` are nullable, allowing for records to exist even if this information is not yet available.
- **Domain Constraint:** The `SERIES.Orientation` column has a specific domain constraint, limiting its values to a predefined set (e.g., 'Sagittal', 'Transverse', 'Box', 'Spine', 'Unknown').
- **Referential Integrity:** As noted, `FOREIGN KEY` constraints guarantee that the relationships between tables (e.g., studies and patients) are never broken.

3 Database Deployment

This section details the practical, Python-based ETL (Extract, Transform, Load) process used to create and populate the `LumbarMRI` database. Unlike a traditional SQL-only deployment, this approach uses Python scripts to connect to SQL Server, dynamically create the necessary schema, and import data from a hierarchical file system and a CSV file.

This deployment process restores the database to its clean, unindexed state, which sets the foundation for the performance optimization analysis conducted in Section 4.

3.1 Prerequisites

Before execution, the following components must be in place:

- **SQL Server:** A running instance of SQL Server, accessible via Windows Authentication.
- **Python Environment:** Python 3.x with the `pyodbc` and `pandas` packages installed.
- **ODBC Driver:** ODBC Driver 17 for SQL Server.
- **Source Data:** The dataset must be available at the path `D:`
  `HK251`
  `Database Management...`

3.2 Phase 1: Manual Database Creation

The Python scripts are designed to connect to an existing database. The blank database must be created manually in SSMS first.

```
CREATE DATABASE LumbarMRI;
GO
```

3.3 Phase 2: Schema Creation and MRI Data Import

This phase uses the `script.py` file to build the core schema and populate it with data from the MRI file system.

- **Script:** `import_mri_data.py`
- **Purpose:** To scan the hierarchical MRI folder structure (`01_MRI_Data`), extract metadata, and populate the database.
- **Execution:**

```
python import_mri_data.py
```

Script Logic:

1. **Connection:** The script connects to the `LumbarMRI` database on the local SQL Server instance.
2. **Schema Creation (Dynamic):** The script executes `IF NOT EXISTS... CREATE TABLE` statements for the four primary tables: `Patients`, `Studies`, `Series`, and `Images`. This step dynamically builds the schema if it doesn't already exist.
3. **ETL Process:**
   - **Extract:** The script traverses the folder structure (`./{PatientID}/{StudyName}/{SeriesName}/...*.ima`).
   - **Transform:** It parses metadata from the folder and file names. For example:
     - It extracts `StudyDate` using a regular expression.
     - It determines `Orientation` (e.g., 'Sagittal', 'Transverse') by analyzing the `SeriesName`.
     - It counts the number of `.ima` files to get the `FileCount`.
   - **Load:** It inserts the transformed data into the respective tables, maintaining referential integrity (e.g., inserting a `Patient` first, then its `Studies`, then their `Series`).

3.4 Phase 3: Clinical Data Import

This phase uses the `import_radiologists_data.py` script to add the fifth table, `RadiologistsData`, and populate it from a CSV file.

- **Script:** `import_radiologists_data.py`
- **Purpose:** To import clinical notes from `RadiologistsData.csv` and link them to `Patient` records.
- **Execution:**

```
python import_radiologists_data.py
```

Script Logic:

1. **Connection:** The script connects to the same `LumbarMRI` database.
2. **Schema Creation (Dynamic):** It executes an `IF NOT EXISTS... CREATE TABLE` statement for the final table, `RadiologistsData`, establishing its `FOREIGN KEY` link to the `Patients` table.
3. **ETL Process:**
   - **Extract:** The script reads `RadiologistsData.csv` into a pandas DataFrame.
   - **Transform (and Load):** It iterates through each row of the DataFrame.
     - It checks if the `PatientID` from the CSV already exists in the `Patients` table.
     - **Integrity Check:** If the patient does not exist, the script creates a new `Patient` record. This is crucial for handling patients who have clinical notes but no corresponding MRI scans (60 such patients were identified).
     - **Load:** It inserts the `PatientID` and the `Clinician's Notes` (handling `NULL` values) into the `RadiologistsData` table.

3.5 Cleanup and Reset

To reset the database for repeatable testing, the entire database can be dropped. The deployment process (Phases 1-3) can then be run again to restore a clean, unindexed state.

```
DROP DATABASE LumbarMRI;
GO
```

4 How DBMS works with indexing

To demonstrate how a DBMS leverages indexing for query optimization, this section presents a series of practical case studies performed on the `LumbarMRI` database. We will analyze the performance of common queries in their default, unoptimized state and then compare the results "before" and "after" implementing an appropriate index.

All performance metrics are captured using `SET STATISTICS IO, TIME ON;` and the **Actual Execution Plan** in SQL Server Management Studio (SSMS).

4.1 Case Study 1: Optimizing a Highly Selective `WHERE` Clause

4.1.1 Scenario and Initial Analysis (Before Optimization)

Scenario: To demonstrate index selectivity, we use the filter `Orientation = 'Box'`. Our analysis confirms that this filter returns only **7 rows** (0.19% of the table), which is crucial as the entire table costs only **42 logical reads** to scan. This highly selective scenario is designed to showcase the optimal usage of an Index Seek.

Query: The query is executed in the unindexed state:

```
SET STATISTICS IO, TIME ON;
SELECT * FROM Series WHERE Orientation = 'Box';
```

Implement 1: Case Study 1: Unoptimized Query (Selective)

Analysis of Unoptimized State: As the `Orientation` column lacks an index, the query optimizer is forced to perform a Clustered Index Scan (reading all 3,761 rows). The `STATISTICS IO` output confirms the I/O cost of scanning the entire table.

Clustered Index Scan (Clustered)

Scanning a clustered index, entirely or only a range.

Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.036308 (100%)
Estimated I/O Cost	0.0320139
Estimated Subtree Cost	0.036308
Estimated CPU Cost	0.0042941
Estimated Number of Executions	1
Estimated Number of Rows to be Read	3761
Estimated Number of Rows for All Executions	7
Estimated Number of Rows Per Execution	7
Estimated Row Size	297 B
Ordered	False
Node ID	0

Predicate

[LumbarMRI].[dbo].[Series].[Orientation]=CONVERT_IMPLICIT(nvarchar(4000),[@1],0)

Object

[LumbarMRI].[dbo].[Series].[PK__Series__F3A1C1018D1E02E5]

Output List

[LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].StudyID, [LumbarMRI].[dbo].[Series].SeriesName, [LumbarMRI].[dbo].[Series].Orientation, [LumbarMRI].[dbo].[Series].FileCount

Figure 3: Execution Plan showing a Clustered Index Scan (Before)

Initial Metrics:

- **Operator:** Clustered Index Scan
- **Logical Reads:** 42
- **Elapsed Time:** 35 ms

4.1.2 Optimization Strategy: Non-Clustered Index

Rationale: The expected cost of a `Seek + Lookup` (approx. 16 reads) is significantly less than the fixed cost of a `Scan` (42 reads). The creation of a dedicated index will now successfully guide the optimizer away from the full table scan.

Solution: We create a non-clustered index on the filtering column.

```
CREATE INDEX IX_Series_Orientation
ON Series(Orientation);
```

Implement 2: Case Study 1: Index Creation

4.1.3 Results and Verification (After Optimization)

Query: The exact same query is executed again after the index creation.

Analysis of Optimized State: The optimizer now correctly identifies that the new index is the cheaper path. The execution plan should show the operation shifting to an Index Seek, followed by Key Lookups to fetch the remaining columns (`SeriesName`, `FileCount`, etc.) requested by `SELECT *`.

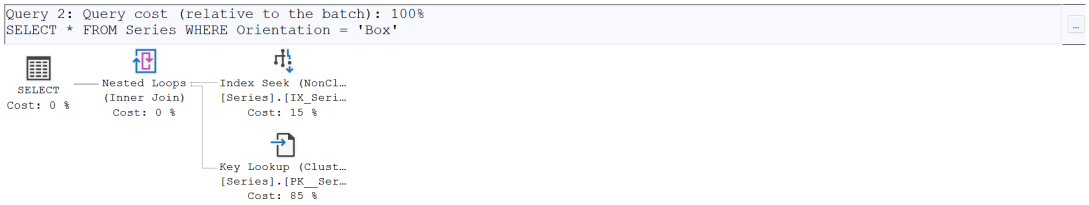


Figure 4: Optimized Plan Showing Index Seek and Costly Key Lookup

Index Seek (NonClustered)

Scan a particular range of rows from a nonclustered index.

Physical Operation	Index Seek
Logical Operation	Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.0032897 (15%)
Estimated I/O Cost	0.003125
Estimated Subtree Cost	0.0032897
Estimated CPU Cost	0.0001647
Estimated Number of Executions	1
Estimated Number of Rows to be Read	7
Estimated Number of Rows for All Executions	7
Estimated Number of Rows Per Execution	7
Estimated Row Size	32 B
Ordered	True
Node ID	1

Object
[LumbarMRI].[dbo].[Series].[IX_Series_Orientation]
Output List
[LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].Orientation
Seek Predicates
Seek Keys[1]: Prefix: [LumbarMRI].[dbo].[Series].Orientation = Scalar Operator(N'Box')

(a) Index Seek (15% Cost): Uses the Non-Clustered Index to quickly find the 7 rows matching `Orientation='Box'`.

Key Lookup (Clustered)

Uses a supplied clustering key to lookup on a table that has a clustered index.

Physical Operation	Key Lookup
Logical Operation	Key Lookup
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	0.003125
Estimated Operator Cost	0.0187232 (85%)
Estimated CPU Cost	0.0001581
Estimated Subtree Cost	0.0187232
Estimated Number of Executions	7
Estimated Number of Rows for All Executions	7
Estimated Number of Rows Per Execution	1
Estimated Row Size	274 B
Ordered	True
Node ID	3

Object
[LumbarMRI].[dbo].[Series].[PK__Series__F3A1C1018D1E02E5]
Output List
[LumbarMRI].[dbo].[Series].StudyID, [LumbarMRI].[dbo].[Series].SeriesName, [LumbarMRI].[dbo].[Series].FileCount
Seek Predicates
Seek Keys[1]: Prefix: [LumbarMRI].[dbo].[Series].SeriesID = Scalar Operator([LumbarMRI].[dbo].[Series].[SeriesID])

(b) Key Lookup (85% Cost): Fetches the remaining columns (`SELECT *`) by accessing the Clustered Index 7 times. This is the primary bottleneck.

Figure 5: Optimized Plan Showing Index Seek and Costly Key Lookup

Performance Metrics: The `STATISTICS IO` output confirms the dramatic I/O reduction.

```
Table 'Series'. Scan count 1, logical reads 16, ...
SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 4 ms.
```

Optimized Metrics:

- **Operator:** Index Seek (+ Key Lookup)
- **Logical Reads:** 16
- **Elapsed Time:** 4 ms

4.1.4 Case Study 1: Conclusion

By targeting a highly selective query, the optimization successfully changed the execution strategy:

Metric	Before (No Index)	After (With Index)	Improvement
Operator	Clustered Index Scan	Index Seek	More Efficient
Logical Reads	42	16	**61.9% Reduction**
Elapsed Time	35 ms	4 ms	**88.6% Reduction**

Table 2: Comparison of Performance Metrics for Case Study 1 (Selective)

This result confirms that an index is only beneficial when the cost of the `Seek` (I/O to traverse the index structure) plus the cost of the `Lookups` (I/O to fetch data) is less than the total cost of the `Scan`.

4.2 Case Study 2: Optimizing Multi-Table Joins on Foreign Keys

4.2.1 Scenario and Initial Analysis (Before Optimization)

Scenario: A fundamental analytical task in a hierarchical medical database is tracing data from the highest level (`Patient`) down to the middle level (`Series`). This query requires joining the `Studies` table (558 records, FK on `PatientID`) and the `Series` table (3,761 records, FK on `StudyID`).

Query: We select all series data for a specific patient ID (e.g., `PatientID = 100`).

```
SET STATISTICS IO, TIME ON;
SELECT
    s.SeriesID, s.SeriesName, s.Orientation
FROM Studies AS st
JOIN Series AS s ON st.StudyID = s.StudyID
WHERE st.PatientID = 100;
SET STATISTICS IO, TIME OFF;
```

Implement 3: Case Study 2: Multi-Join Query (Unoptimized)

Analysis of Unoptimized State: The default schema does not index Foreign Key columns. Consequently, the optimizer cannot efficiently execute the join.

- **Bottleneck 1:** Finding `Studies` for `PatientID = 100` requires a Clustered Index Scan on the entire `Studies` table.
- **Bottleneck 2:** The subsequent join to `Series` on `StudyID` is also performed inefficiently, likely using another Clustered Index Scan or a costly Hash Match/Merge Join operation.
- **I/O Cost:** The total Logical Reads will be the sum of scanning a large portion of both tables.

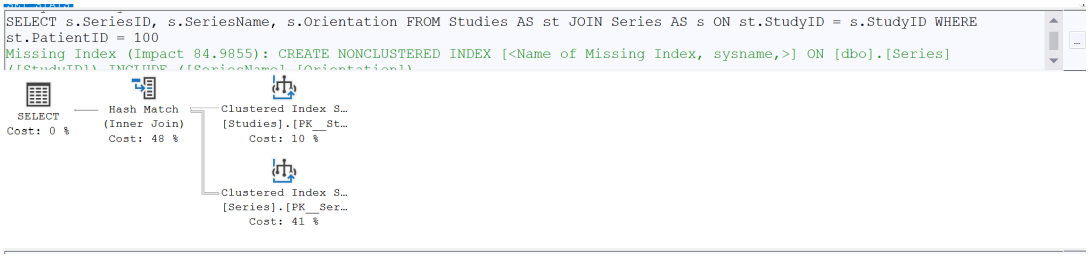
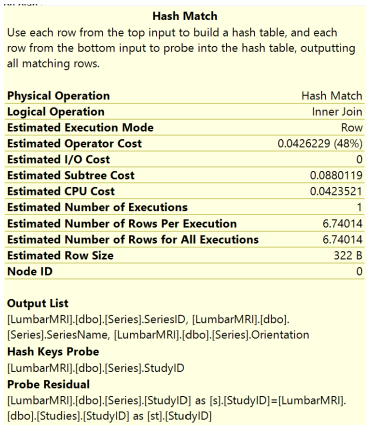


Figure 6: Execution Plan showing Clustered Index Scans and inefficient Hash/Merge Join (Before)



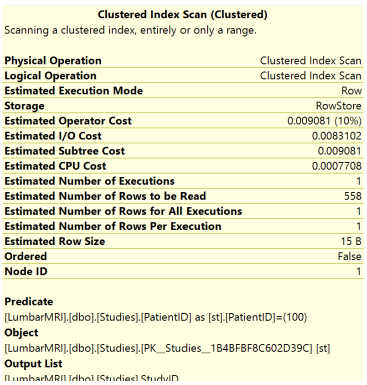
Hash Match

Use each row from the top input to build a hash table, and each row from the bottom input to probe into the hash table, outputting all matching rows.

Physical Operation	Hash Match
Logical Operation	Inner Join
Estimated Execution Mode	Row
Estimated Operator Cost	0.0426229 (48%)
Estimated I/O Cost	0
Estimated Subtree Cost	0.0880119
Estimated CPU Cost	0.0423521
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	6.74014
Estimated Number of Rows for All Executions	6.74014
Estimated Row Size	322 B
Node ID	0

Output List
[LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].SeriesName, [LumbarMRI].[dbo].[Series].Orientation
Hash Keys Probe
[LumbarMRI].[dbo].[Series].StudyID
Probe Residual
[LumbarMRI].[dbo].[Series].[StudyID] as [s].[StudyID]=[LumbarMRI].[dbo].[Studies].[StudyID] as [st].[StudyID]

(a) Hash Match (48% Cost): The optimizer's chosen join operator. It builds a hash table in memory from one input (the Studies scan) and probes it with the other (the Series scan). This is an expensive operation, used when no suitable indexes are available for a faster Nested Loops Join.



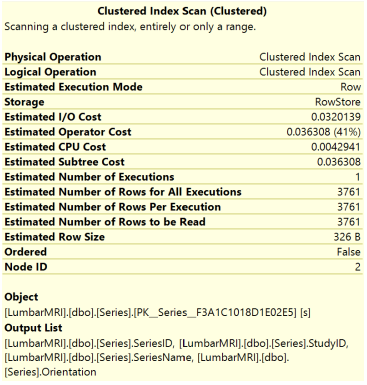
Clustered Index Scan (Clustered)

Scanning a clustered index, entirely or only a range.

Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.009081 (10%)
Estimated I/O Cost	0.0083102
Estimated Subtree Cost	0.009081
Estimated CPU Cost	0.0007708
Estimated Number of Executions	1
Estimated Number of Rows to be Read	558
Estimated Number of Rows for All Executions	1
Estimated Number of Rows Per Execution	1
Estimated Row Size	15 B
Ordered	False
Node ID	1

Predicate
[LumbarMRI].[dbo].[Studies].[PatientID] as [st].[PatientID]=(100)
Object
[LumbarMRI].[dbo].[Studies].[PK__Studies__1B4BFBF8C602D39C] [st]
Output List
[LumbarMRI].[dbo].[Studies].StudyID

(b) Clustered Index Scan (10% Cost): The first bottleneck. To satisfy the WHERE st.PatientID = 100 clause, the engine must scan the entire Studies table (558 rows) because the PatientID foreign key column is not indexed.



Clustered Index Scan (Clustered)

Scanning a clustered index, entirely or only a range.

Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	0.0320139
Estimated Operator Cost	0.036308 (41%)
Estimated CPU Cost	0.0042941
Estimated Subtree Cost	0.036308
Estimated Number of Executions	1
Estimated Number of Rows for All Executions	3761
Estimated Number of Rows Per Execution	3761
Estimated Number of Rows to be Read	3761
Estimated Row Size	326 B
Ordered	False
Node ID	2

Object
[LumbarMRI].[dbo].[Series].[PK__Series__F3A1C1018D1E02E5] [s]
Output List
[LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].StudyID, [LumbarMRI].[dbo].[Series].SeriesName, [LumbarMRI].[dbo].[Series].Orientation

(c) Clustered Index Scan (41% Cost): The second bottleneck. The engine must also scan the entire Series table (3761 rows) to provide the data for the Hash Match operator, as the StudyID join column is also not indexed.

Figure 7: Execution Plan showing Clustered Index Scans and inefficient Hash/Merge Join (Before)

4.2.2 Optimization Strategy: Non-Clustered Indexes on Foreign Keys

Rationale: Foreign Key columns are the primary connection points between tables. Indexing them is crucial to enable the optimizer to use efficient Index Seek operations for both the filtering (`WHERE`) and the joining (`ON`) clauses. This allows the DBMS to use the fast Nested Loops Join operator.

Solution: We create two non-clustered indexes on the critical FK columns used in the join path.

```
CREATE INDEX IX_Studies_PatientID
ON Studies(PatientID);
CREATE INDEX IX_Series_StudyID
ON Series(StudyID);
```

Implement 4: Case Study 2: Index Creation

4.2.3 Results and Verification (After Optimization)

Query: The exact same query is executed again after the indexes have been created.

Analysis of Optimized State: The plan is expected to transition from costly scans to highly efficient seeks.

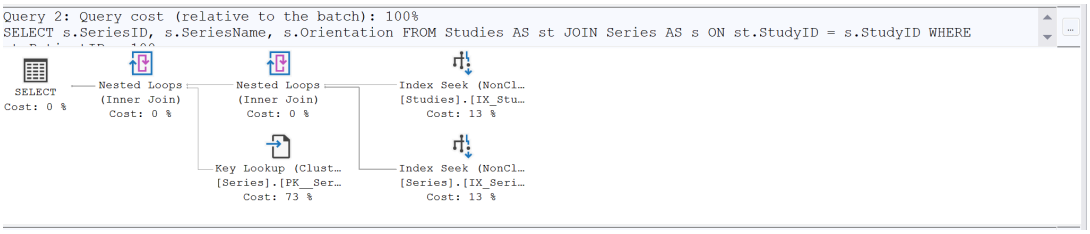


Figure 8: Execution Plan showing Index Seeks connected by Nested Loops Join (After)

Key Lookup (Clustered)
Uses a supplied clustering key to lookup on a table that has a clustered index.

Physical Operation	Key Lookup
Logical Operation	Key Lookup
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	0.003125
Estimated Operator Cost	0.0179733 (73%)
Estimated CPU Cost	0.0001581
Estimated Subtree Cost	0.0179733
Estimated Number of Executions	6.74014
Estimated Number of Rows for All Executions	6.74014
Estimated Number of Rows Per Execution	1
Estimated Row Size	318 B
Ordered	True
Node ID	5

Object
[LumbarMRI].[dbo].[Series].[PK__Series__F3A1C1018D1E02E5] [s]
Output List
[LumbarMRI].[dbo].[Series].SeriesName, [LumbarMRI].[dbo].[Series].Orientation
Seek Predicates
Seek Keys[1]: Prefix: [LumbarMRI].[dbo].[Series].SeriesID = Scalar Operator([LumbarMRI].[dbo].[Series].[SeriesID] as [s].[SeriesID])

(a) Key Lookup (73% Cost): This is the new primary bottleneck. Because the SELECT list asks for columns (SeriesName, Orientation) that are not in the IX_Series_StudyID index, the engine must perform a separate lookup on the main Series clustered index for every row found (6.7 executions). This high cost motivates Case Study 3.

Index Seek (NonClustered)
Scan a particular range of rows from a nonclustered index.

Physical Operation	Index Seek
Logical Operation	Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.0032831 (13%)
Estimated I/O Cost	0.003125
Estimated Subtree Cost	0.0032831
Estimated CPU Cost	0.0001581
Estimated Number of Executions	1
Estimated Number of Rows to be Read	1
Estimated Number of Rows for All Executions	1
Estimated Number of Rows Per Execution	1
Estimated Row Size	11 B
Ordered	True
Node ID	2

Object
[LumbarMRI].[dbo].[Studies].[IX_Studies_PatientID] [st]
Output List
[LumbarMRI].[dbo].[Studies].StudyID
Seek Predicates
Seek Keys[1]: Prefix: [LumbarMRI].[dbo].[Studies].PatientID = Scalar Operator((100))

(b) Index Seek (13% Cost): The first efficient operation. The optimizer uses the new IX_Studies_PatientID index to instantly find the records for PatientID = 100. This completely replaces the 10% cost Clustered Index Scan from the "Before" plan.

Index Seek (NonClustered)
Scan a particular range of rows from a nonclustered index.

Physical Operation	Index Seek
Logical Operation	Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.0032894 (13%)
Estimated I/O Cost	0.003125
Estimated Subtree Cost	0.0032894
Estimated CPU Cost	0.0001644
Estimated Number of Executions	1
Estimated Number of Rows to be Read	6.74014
Estimated Number of Rows for All Executions	6.74014
Estimated Number of Rows Per Execution	6.74014
Estimated Row Size	11 B
Ordered	True
Node ID	3

Object
[LumbarMRI].[dbo].[Series].[IX_Series_StudyID] [s]
Output List
[LumbarMRI].[dbo].[Series].SeriesID
Seek Predicates
Seek Keys[1]: Prefix: [LumbarMRI].[dbo].[Series].StudyID = Scalar Operator([LumbarMRI].[dbo].[Studies].[StudyID] as [st].[StudyID])

(c) Index Seek (13% Cost): The second efficient operation. For each row found by the first seek, the Nested Loops Join uses the new IX_Series_StudyID index to perform a fast seek on the Series table. This replaces the 41% cost Clustered Index Scan from the "Before" plan.

Figure 9: Execution Plan showing efficient Index Seeks and a new Key Lookup bottleneck (After)

Performance Metrics: The `STATISTICS IO` output confirms the massive reduction in I/O across both tables. The costly scans are replaced by highly targeted seeks.

```
-- "Before" Optimization (Unindexed)
Table 'Series'. Scan count 1, logical reads 42, ...
Table 'Studies'. Scan count 1, logical reads 10, ...

SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 4 ms.

-- "After" Optimization (FKs Indexed)
(6 rows affected)
Table 'Series'. Scan count 1, logical reads 14, ...
Table 'Studies'. Scan count 1, logical reads 2, ...

SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```

Initial Metrics (Before):

- **Primary Bottleneck:** Clustered Index Scans + Hash Join
- **Total Logical Reads: 52** (42 + 10)
- **Elapsed Time: 4 ms**

Optimized Metrics (After):

- **Operator:** Index Seeks + Nested Loops Join
- **Total Logical Reads: 16** (14 + 2)
- **Elapsed Time: 0 ms**

4.2.4 Case Study 2: Conclusion

Indexing the foreign key columns provided a fundamental improvement to the query plan, transforming the join strategy from inefficient `Scans` to highly efficient `Seeks`.

Metric	Before (No FK Index)	After (FK Indexed)	Improvement
Join Operator	Hash Match	Nested Loops	More Efficient
Total Logical Reads	**52**	**16**	**69.2% Reduction**
Elapsed Time	**4 ms**	**0 ms**	**100% Reduction**

Table 3: Comparison of Performance Metrics for Case Study 2

Key Finding for Next Steps: The plan is now highly efficient at joining. However, the `Key Lookup (73% Cost)` on the `Series` table (identified in Figure 8a) has become the new bottleneck. This is because the query selected `SeriesName` and `Orientation`, which were not included in the `IX_Series_StudyID` index. This observation directly motivates the final optimization in Case Study 3.

4.3 Case Study 3: The Advanced Win - Eliminating Key Lookups

4.3.1 Scenario and Initial Analysis (Before Optimization)

Scenario: Case Study 2 was a success in optimizing the `JOIN` operation, reducing I/O by 69.2% and changing the plan from `Scans` to `Seeks`. However, the optimized plan (Figure 8) revealed a new, dominant bottleneck: a `Key Lookup` operation on the `Series` table, consuming 73% of the query cost.

Analysis of "Before" State (The new bottleneck): This `Key Lookup` occurs because:

1. Our query selects `s.SeriesID, s.SeriesName, s.Orientation`.
2. Our index (`IX_Series_StudyID`) only contains the `StudyID` column.
3. For every row found by the `Index Seek`, the DBMS must perform a costly "lookup" back to the main clustered index to retrieve the three columns from the `SELECT` list.

This section's goal is to eliminate this final bottleneck.

Initial Metrics (From Case Study 2 "After"):

- **Primary Bottleneck: Key Lookup (73% Cost)**
- **Total Logical Reads: 16** (Studies: 2, Series: 14)
- **Elapsed Time: 0 ms**

4.3.2 Optimization Strategy: Covering Index

Rationale: To eliminate the `Key Lookup`, we must create an index that "covers" the query. This means the index itself must contain all the columns required by the query, so the DBMS never has to touch the main table. We can achieve this by using the `INCLUDE` clause to add the non-key columns (`SeriesID, SeriesName, Orientation`) to the index's leaf level.

Solution: We will drop the non-covering index on `Series` and replace it with a new non-clustered index that "covers" all required columns. The index on `Studies` is already optimal.

```
-- 1. Drop the old, inefficient index from Case Study 2
DROP INDEX IX_Series_StudyID ON Series;

-- 2. Create the new Covering Index
CREATE INDEX IX_Series_StudyID_Cover
ON Series(StudyID) -- Column for JOIN (Key)
INCLUDE (SeriesID, SeriesName, Orientation); -- Columns for SELECT (Cover)
```

Implement 5: Case Study 3: Creating a Covering Index

4.3.3 Results and Verification (After Optimization)

Query: The exact same query is executed again after the new covering index is in place.

```
SET STATISTICS IO, TIME ON;
SELECT
    s.SeriesID, s.SeriesName, s.Orientation
FROM Studies AS st
JOIN Series AS s ON st.StudyID = s.StudyID
WHERE st.PatientID = 100;
SET STATISTICS IO, TIME OFF;
```

Implement 6: Case Study 3: Final Optimized Query

Analysis of Optimized State: The new execution plan (Figure 10) shows the final, optimal form. The costly `Key Lookup (73%)` has been completely eliminated. The query is now resolved entirely by seeking on two efficient indexes. The I/O on the `Series` table dropped from 14 reads to only 2, as the engine no longer reads from the clustered index, reading only from the new, smaller, covering index pages.

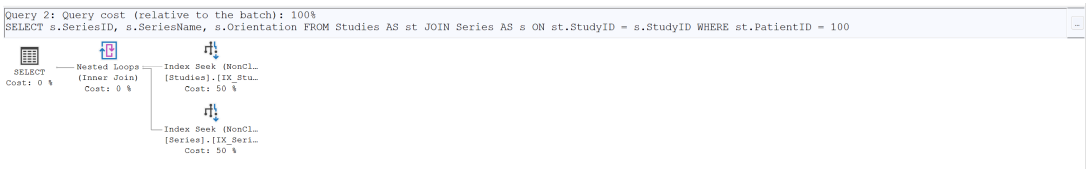


Figure 10: Final Execution Plan showing two Index Seeks and NO Key Lookup (After)

Index Seek (NonClustered)

Scan a particular range of rows from a nonclustered index.

Physical Operation	Index Seek
Logical Operation	Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.0032831 (50%)
Estimated I/O Cost	0.003125
Estimated Subtree Cost	0.0032831
Estimated CPU Cost	0.0001581
Estimated Number of Executions	1
Estimated Number of Rows to be Read	1
Estimated Number of Rows for All Executions	1
Estimated Number of Rows Per Execution	1
Estimated Row Size	11 B
Ordered	True
Node ID	1

Object
[LumbarMRI].[dbo].[Studies].[IX_Studies_PatientID] [st]
Output List
[LumbarMRI].[dbo].[Studies].StudyID
Seek Predicates
Seek Keys[1]: Prefix: [LumbarMRI].[dbo].[Studies].PatientID = Scalar Operator((100))

(a) Index Seek (50% Cost): This first seek efficiently uses the IX_Studies_PatientID index to find the study records for PatientID = 100. This operation is unchanged from Case Study 2.

Index Seek (NonClustered)

Scan a particular range of rows from a nonclustered index.

Physical Operation	Index Seek
Logical Operation	Index Seek
Estimated Execution Mode	Row
Storage	RowStore
Estimated Operator Cost	0.0032894 (50%)
Estimated I/O Cost	0.003125
Estimated Subtree Cost	0.0032894
Estimated CPU Cost	0.0001644
Estimated Number of Executions	1
Estimated Number of Rows to be Read	6.74014
Estimated Number of Rows for All Executions	6.74014
Estimated Number of Rows Per Execution	6.74014
Estimated Row Size	322 B
Ordered	True
Node ID	2

Object
[LumbarMRI].[dbo].[Series].[IX_Series_StudyID_Cover] [s]
Output List
[LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].SeriesName, [LumbarMRI].[dbo].[Series].Orientation
Seek Predicates
Seek Keys[1]: Prefix: [LumbarMRI].[dbo].[Series].StudyID = Scalar Operator([LumbarMRI].[dbo].[Studies].[StudyID] as [st].[StudyID])

(b) Index Seek (50% Cost): This second seek uses the new IX_Series_StudyID_Cover index. The Output List confirms that it retrieves SeriesID, SeriesName, and Orientation directly from the index. This completely eliminates the 73% Key Lookup bottleneck from the previous case study.

Figure 11: Final Optimized Plan Showing Two Efficient Index Seeks and No Key Lookup

Performance Metrics: The `STATISTICS IO` output shows the final, minimal I/O cost.

```
-- "After" Optimization (Covering Index)
(6 rows affected)
Table 'Series'. Scan count 1, logical reads 2, ...
Table 'Studies'. Scan count 1, logical reads 2, ...

SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
```

Optimized Metrics (Final):

- **Primary Bottleneck:** (None - Fully Optimized)
- **Total Logical Reads: 4** (Studies: 2, Series: 2)
- **Elapsed Time: 0 ms**

4.3.4 Case Study 3: Conclusion

The final optimization step, replacing the `Key Lookup` with a `Covering Index`, resolved the last remaining bottleneck. This demonstrates an advanced optimization technique where the index leaf nodes store all data needed for the query.

Metric	Before (CS2 After)	After (Covering Index)	Improvement
Bottleneck	Key Lookup (73% Cost)	(None)	Bottleneck Eliminated
Total Logical Reads	**16**	**4**	**75.0% Reduction**
Elapsed Time	**0 ms**	**0 ms**	No Change

Table 4: Comparison of Performance Metrics for Case Study 3

This three-stage analysis demonstrates the iterative process of database optimization:

1. **CS1:** Proved that a `Seek` is better than a `Scan`.
2. **CS2:** Proved that indexing `FKs` is critical for `JOINs`.
3. **CS3:** Proved that a `Covering Index` is the final step to eliminate I/O bottlenecks from `SELECT` lists.

4.4 Case study 4: Columnstore Index for Analytical Queries (SQL Server Specific)

4.4.1 Scenerio and Initial Analysis (Before Optimizing)

Scenario: While previous case studies focused on transactional queries (OLTP), real-world systems often run analytical queries (OLAP) to summarize data. We execute a complex aggregation query that joins four tables (`Patients`, `Studies`, `Series`, `Images`) to calculate statistics per patient (e.g., Total Images, Avg Files per Series).

Query: The following query is executed without any columnstore indexes:

```
SET STATISTICS IO, TIME ON;
GO
SELECT
    p.PatientID,
    COUNT(DISTINCT st.StudyID) AS TotalStudies,
    COUNT(DISTINCT ser.SeriesID) AS TotalSeries,
    COUNT(i.ImageID) AS TotalImages,
    AVG(ser.FileCount) AS AvgFilesPerSeries
FROM Patients p
INNER JOIN Studies st ON p.PatientID = st.PatientID
INNER JOIN Series ser ON st.StudyID = ser.StudyID
INNER JOIN Images i ON ser.SeriesID = i.SeriesID
GROUP BY p.PatientID
ORDER BY TotalImages DESC;
GO
```

Implement 7: Case Study 4: Column stored (Unoptimized)

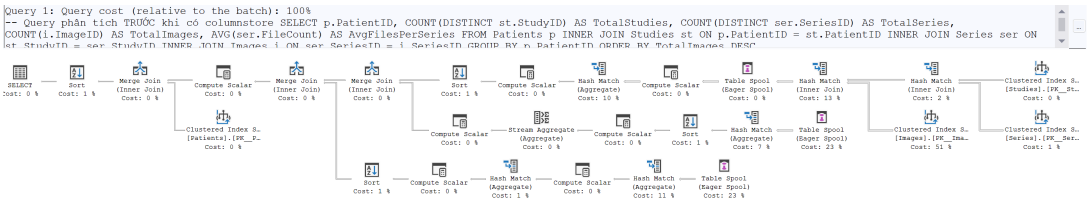
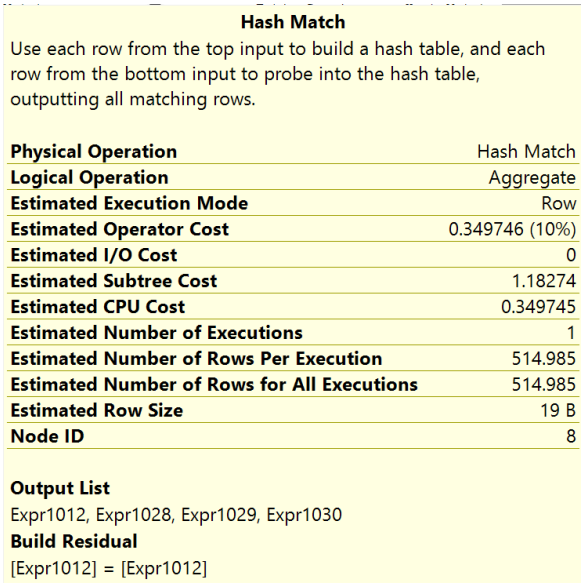


Figure 12: Execution Plan showing (Before)

Analysis of Unoptimized State (Row-Store): The default Row-Store architecture is inefficient for this type of wide aggregation. The Execution Plan reveals severe bottlenecks caused by Row Mode execution:

- **Clustered Index Scan (51% Cost):** To count images, the engine scans the entire `Images` table (48,345 rows) page-by-page. In Row-Store, this forces the engine to read unnecessary columns into memory, wasting I/O bandwidth.
- **Table Spool (23% Cost):** Due to the complexity of joining and aggregating distinct values in Row Mode, the engine is forced to spool intermediate results to a temporary table (TempDB), adding significant latency.
- **Hash Match (Aggregates/Joins):** These operators process data row-by-row (Row Execution Mode), which consumes high CPU cycles for large dataset aggregations.



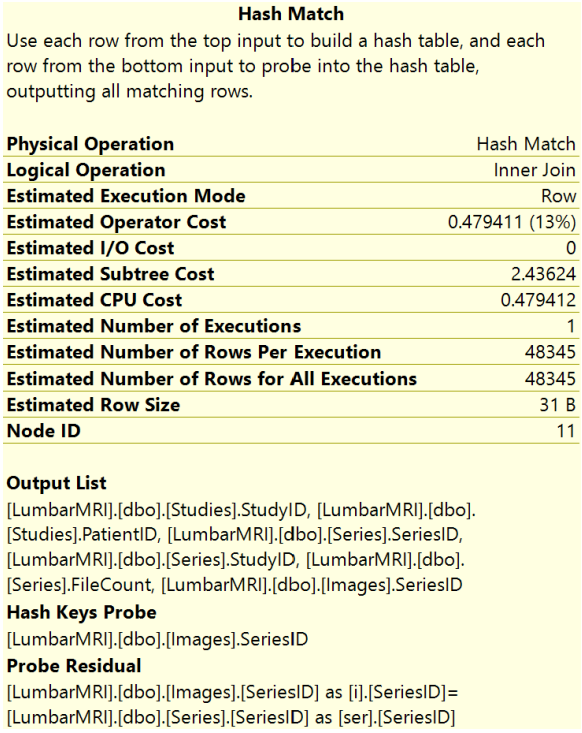
Hash Match

Use each row from the top input to build a hash table, and each row from the bottom input to probe into the hash table, outputting all matching rows.

Physical Operation	Hash Match
Logical Operation	Aggregate
Estimated Execution Mode	Row
Estimated Operator Cost	0.349746 (10%)
Estimated I/O Cost	0
Estimated Subtree Cost	1.18274
Estimated CPU Cost	0.349745
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	514.985
Estimated Number of Rows for All Executions	514.985
Estimated Row Size	19 B
Node ID	8

Output List
Expr1012, Expr1028, Expr1029, Expr1030
Build Residual
[Expr1012] = [Expr1012]

(a) Hash match (Aggregate) 10%



Hash Match

Use each row from the top input to build a hash table, and each row from the bottom input to probe into the hash table, outputting all matching rows.

Physical Operation	Hash Match
Logical Operation	Inner Join
Estimated Execution Mode	Row
Estimated Operator Cost	0.479411 (13%)
Estimated I/O Cost	0
Estimated Subtree Cost	2.43624
Estimated CPU Cost	0.479412
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	48345
Estimated Number of Rows for All Executions	48345
Estimated Row Size	31 B
Node ID	11

Output List
[LumbarMRI].[dbo].[Studies].StudyID, [LumbarMRI].[dbo].[Studies].PatientID, [LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].StudyID, [LumbarMRI].[dbo].[Series].FileCount, [LumbarMRI].[dbo].[Images].SeriesID
Hash Keys Probe
[LumbarMRI].[dbo].[Images].SeriesID
Probe Residual
[LumbarMRI].[dbo].[Images].[SeriesID] as [i].[SeriesID]= [LumbarMRI].[dbo].[Series].[SeriesID] as [ser].[SeriesID]

(b) Hash match (Inner Join) 13%

Figure 13: Detailed view of costly Hash Match operators running in Row Mode (Before)

Table Spool

Stores the data from the input into a temporary table in order to optimize rewinds.

Physical Operation	Table Spool
Logical Operation	Eager Spool
Estimated Execution Mode	Row
Estimated Operator Cost	0.828159 (23%)
Estimated I/O Cost	0.0044062
Estimated Subtree Cost	0.828159
Estimated CPU Cost	0.0058764
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	48345
Estimated Number of Rows for All Executions	48345
Estimated Row Size	31 B
Node ID	27
Primary Node ID	10

Output List

[LumbarMRI].[dbo].[Studies].StudyID, [LumbarMRI].[dbo].[Studies].PatientID, [LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].StudyID, [LumbarMRI].[dbo].[Series].FileCount, [LumbarMRI].[dbo].[Images].SeriesID

(a) Table Spool 23%

Clustered Index Scan (Clustered)

Scanning a clustered index, entirely or only a range.

Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Estimated Execution Mode	Row
Storage	RowStore
Estimated I/O Cost	1.80238
Estimated Operator Cost	1.85572 (51%)
Estimated CPU Cost	0.0533365
Estimated Subtree Cost	1.85572
Estimated Number of Executions	1
Estimated Number of Rows for All Executions	48345
Estimated Number of Rows Per Execution	48345
Estimated Number of Rows to be Read	48345
Estimated Row Size	11 B
Ordered	False
Node ID	15

Object

[LumbarMRI].[dbo].[Images].[PK__Images__7516F4EC3BF2B55C] [i]

Output List

[LumbarMRI].[dbo].[Images].SeriesID

(b) Clustered Index Scan 51%

Figure 14: Detailed view of primary bottlenecks: Table Spool and Clustered Index Scan (Before)

Initial Metrics (Row-Store): The `STATISTICS IO` output reveals a critical insight: distinct from the data tables, the engine generated massive I/O on an internal `Worktable`.

```
Table 'Worktable'. Scan count 3, logical reads 98656.
Table 'Images'. Scan count 1, logical reads 2440.
SQL Server Execution Times:
   CPU time = 235 ms,  elapsed time = 357 ms.
```

- **Execution Mode:** Row Mode (High Overhead)
- **Primary Data Bottleneck:** Scanning the `Images` table (2,440 logical reads).
- **Hidden Processing Bottleneck:** The **98,656 logical reads** on the `Worktable` confirm the high cost of the **Table Spool** operator, proving that Row Mode is struggling to handle the aggregation logic in memory.
- **Elapsed Time:** 357 ms.

4.4.2 OPTIMIZATION: Create Nonclustered Columnstore Index

Rationale: To optimize analytical queries involving aggregations on large datasets, we need to switch from a row-based processing model to a column-based one. SQL Server's Columnstore Index offers two critical advantages:

1. **Columnar Storage:** Stores data by column rather than by row, allowing high compression (up to 10x) and fetching only the specific columns needed for the query.
2. **Batch Mode Execution:** Processes rows in batches (typically 900 rows at a time) rather than one by one, significantly reducing CPU overhead for aggregation operators like `Hash Match`.

Solution: We create a Non-Clustered Columnstore Index (NCCI) on the `Images` and `Series` tables. These indexes will cover the analytical columns and trigger Batch Mode execution.

```
-- Create columnstore index on Images table for analytical queries
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Images_Analytics
ON Images (SeriesID, FileName)
```

```
WITH (DATA_COMPRESSION = COLUMNSTORE);
GO

-- Alternative: Create columnstore on Series for broader analytics
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Series_Analytics
ON Series (StudyID, SeriesName, Orientation, FileCount)
WITH (DATA_COMPRESSION = COLUMNSTORE);
GO
```

Implement 8: Case Study 4: Column stored (Unoptimized)

4.4.3 Results and Verification (After Optimization)

Query: The exact same analytical query is executed again.

Analysis of Optimized State (Batch Mode): The Execution Plan (Figure 15) reveals a fundamental architectural shift compared to the "Before" state.

- **streamlined Flow:** The expensive **Table Spool** operator has completely disappeared. The plan is now a linear flow, indicating that the engine can process the aggregation in memory without spilling to disk.
- **Columnstore Scans:** The inefficient `Clustered Index Scan` on `Images` and `Series` has been replaced by `Columnstore Index Scan` operators (Cost 10% and 4%), which read compressed column segments.
- **Batch Execution:** The `Hash Match` operators (both Join and Aggregate) now operate in **Batch Mode**, utilizing CPU vectorization to process rows in chunks.

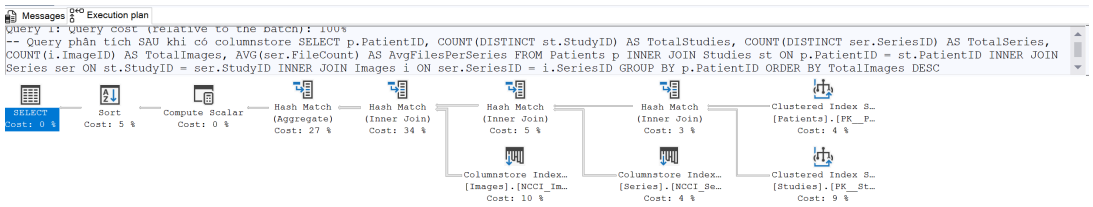


Figure 15: Execution Plan showing (After)

Hash Match
Use each row from the top input to build a hash table, and each row from the bottom input to probe into the hash table, outputting all matching rows.

Physical Operation	Hash Match
Logical Operation	Aggregate
Estimated Execution Mode	Batch
Estimated Operator Cost	0.0274684 (27%)
Estimated I/O Cost	0
Estimated Subtree Cost	0.0956703
Estimated CPU Cost	0.0274684
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	514.985
Estimated Number of Rows for All Executions	514.985
Estimated Row Size	27 B
Node ID	2

Output List
[LumbarMRI].[dbo].[Patients].PatientID, Expr1017, Expr1018, Expr1019, Expr1020, Expr1021

(a) Hash match (Aggregate) 27%

Hash Match
Use each row from the top input to build a hash table, and each row from the bottom input to probe into the hash table, outputting all matching rows.

Physical Operation	Hash Match
Logical Operation	Inner Join
Estimated Execution Mode	Batch
Estimated Operator Cost	0.0345166 (34%)
Estimated I/O Cost	0
Estimated Subtree Cost	0.0682019
Estimated CPU Cost	0.0345135
Estimated Number of Executions	1
Estimated Number of Rows Per Execution	53179.5
Estimated Number of Rows for All Executions	53179.5
Estimated Row Size	23 B
Node ID	3

Output List
[LumbarMRI].[dbo].[Patients].PatientID, [LumbarMRI].[dbo].[Studies].StudyID, [LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].FileCount
Hash Keys Probe
[LumbarMRI].[dbo].[Images].SeriesID
Probe Residual
[LumbarMRI].[dbo].[Images].[SeriesID] as [i].[SeriesID]= [LumbarMRI].[dbo].[Series].[SeriesID] as [ser].[SeriesID]

(b) Hash match (Inner Join) 34%

Figure 16: Detailed view of Batch Mode Hash Match operators (After)

Columnstore Index Scan (NonClustered)
Scan a columnstore index, entirely or only a range.

Physical Operation	Columnstore Index Scan
Logical Operation	Index Scan
Estimated Execution Mode	Batch
Storage	ColumnStore
Estimated Operator Cost	0.0099401 (10%)
Estimated I/O Cost	0.0046065
Estimated Subtree Cost	0.0099401
Estimated CPU Cost	0.0053337
Estimated Number of Executions	1
Estimated Number of Rows to be Read	48345
Estimated Number of Rows for All Executions	48345
Estimated Number of Rows Per Execution	48345
Estimated Row Size	23 B
Ordered	False
Node ID	13

Object
[LumbarMRI].[dbo].[Images].[NCCI_Images_Analytics] [i]
Output List
[LumbarMRI].[dbo].[Images].ImageID, [LumbarMRI].[dbo].[Images].SeriesID, Generation1014

(a) Columnstore Index Scan (NonClustered) 10%

Columnstore Index Scan (NonClustered)
Scan a columnstore index, entirely or only a range.

Physical Operation	Columnstore Index Scan
Logical Operation	Index Scan
Estimated Execution Mode	Batch
Storage	ColumnStore
Estimated Operator Cost	0.0035544 (4%)
Estimated I/O Cost	0.003125
Estimated Subtree Cost	0.0035544
Estimated CPU Cost	0.0004294
Estimated Number of Executions	1
Estimated Number of Rows to be Read	3761
Estimated Number of Rows for All Executions	3761
Estimated Number of Rows Per Execution	3761
Estimated Row Size	27 B
Ordered	False
Node ID	10

Object
[LumbarMRI].[dbo].[Series].[NCCI_Series_Analytics] [ser]
Output List
[LumbarMRI].[dbo].[Series].SeriesID, [LumbarMRI].[dbo].[Series].StudyID, [LumbarMRI].[dbo].[Series].FileCount, Generation1011

(b) Columnstore Index Scan (NonClustered) 4%

Figure 17: Detailed view of Columnstore Index Scans reading compressed segments (After)

Performance Metrics: The `STATISTICS IO` output confirms the efficiency. The most dramatic improvement is the **total elimination of I/O on the Worktable** (Table Spool), proving that the engine no longer needs to spill data to tempdb during aggregation.

```
-- "After" Optimization Statistics
Table 'Worktable'. Scan count 0, logical reads 0.
Table 'Images'. Segment reads 1, lob logical reads 64...
```

```
Table 'Series'. Segment reads 1, lob logical reads 21...

SQL Server Execution Times:
   CPU time = 31 ms,  elapsed time = 44 ms.
```

Optimized Metrics:

- **Execution Mode: Batch Mode** (Vectorized processing).
- **Worktable I/O: 0 reads** (Reduced from 98,656 reads in Row Mode).
- **CPU Time: 31 ms** (Reduced from 235 ms).
- **Elapsed Time: 44 ms** (Reduced from 357 ms).

4.4.4 Case Study 4: Conclusion

By implementing Columnstore Indexes, we unlocked SQL Server's **Batch Mode** processing capabilities. This case study demonstrates that for analytical workloads (OLAP) involving heavy aggregations on large tables, standard Row-Store B-Trees are insufficient. Specialized Columnar storage is required to minimize I/O through compression and maximize CPU efficiency through vectorization.

Metric	Before (Row-Store)	After (Columnstore)	Improvement
Execution Mode	Row Mode	**Batch Mode**	Vectorized Processing
Worktable Reads	**98,656**	**0**	**100% Reduction**
CPU Time	**235 ms**	**31 ms**	**86.8% Reduction**
Elapsed Time	357 ms	44 ms	87.7% Reduction

Table 5: Comparison of Performance Metrics for Case Study 4

5 How DBMS works with query processing

Query processing is the complex procedure of translating high-level, declarative SQL statements into efficient low-level physical operations.This process is not instantaneous; it follows a strict pipeline designed to minimize optimization overhead while maximizing execution speed.

5.1 The Query Optimization Pipeline

As illustrated in Figure 18, the SQL Server Query Optimizer follows a standardized "funnel" approach. The goal is to progressively refine the query plan, starting from logical transformations and ending with physical algorithm selection.

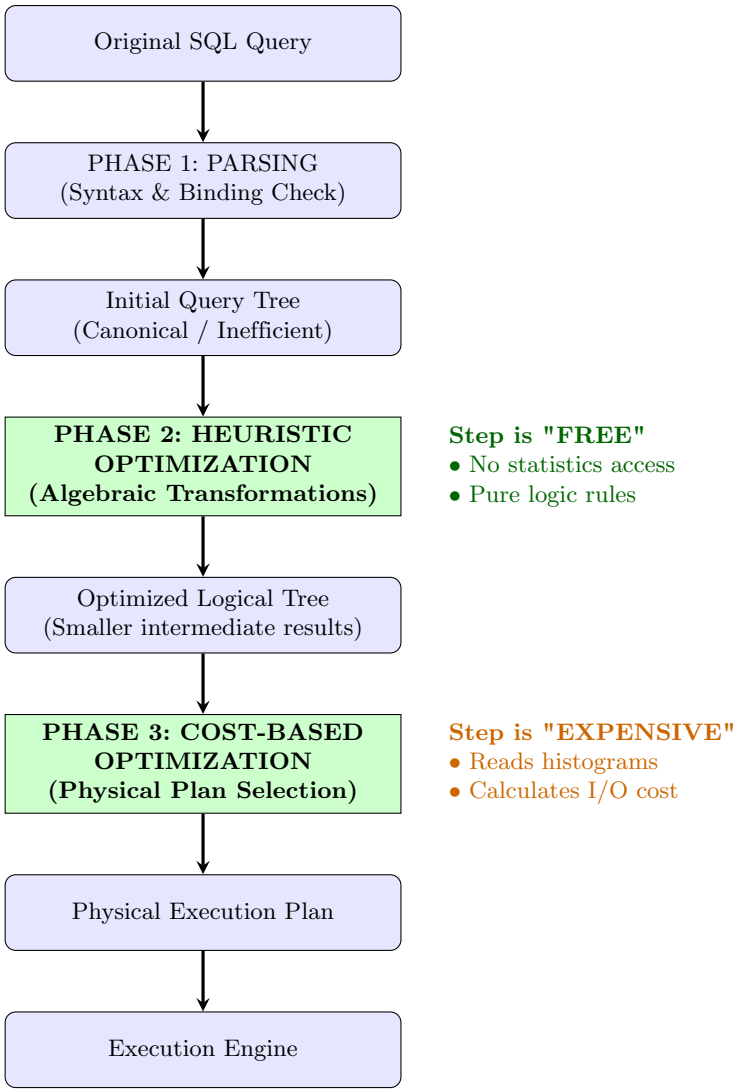


Figure 18: The Complete Query Optimization Pipeline: From SQL to Execution

5.1.1 Why This Specific Order?

The architecture places Heuristics (Phase 2) before Cost-Based Optimization (Phase 3) for strategic reasons:

1. **Heuristics reduce the search space:** The "Canonical" tree generated by the parser is often extremely inefficient (e.g., Cartesian products). If the Cost-Based Optimizer had to calculate costs

for every possible permutation of this raw tree, it would take too long. Heuristics prune the bad options early.

2. **Logic is "Cheap", Physics is "Expensive":**
   - **Heuristic rules** (like "filter early") are algebraic axioms. Applying them requires no disk I/O and no complex math. It is a "free" optimization.
   - **Cost estimation** requires accessing database statistics (histograms), estimating row counts, and calculating CPU/IO formulas. This is computationally expensive.
3. **Efficiency Principle:** It is always better to calculate the cost of a *small, filtered* dataset (after heuristics) than a *large, raw* dataset.

The following sections detail how these phases applied to our specific Case Studies.

5.2 Phase 2: Heuristic Optimization (Logical Plan)

Before calculating costs or considering indexes, the Query Optimizer applies **Heuristic Rules** to transform the initial "canonical" query tree into a more efficient logical form. The core principle is to apply operations that **reduce the size of intermediate results** (SELECT σ, PROJECT π) as early as possible.

5.2.1 Case Study Application: Query Tree Transformation

To demonstrate this, we analyze the logical transformation of our Case Study 2 query:

```
SELECT s.SeriesID, s.SeriesName, s.Orientation
FROM Studies AS st
JOIN Series AS s ON st.StudyID = s.StudyID
WHERE st.PatientID = 100;
```

1. Initial State (Canonical Tree)

Figure 19 shows the tree generated directly from the parser. The `JOIN` operation is placed at the bottom, meaning the engine would theoretically join all 558 studies with 3,761 series ($558 \times 3,761 \approx 2$ million comparisons) before filtering.

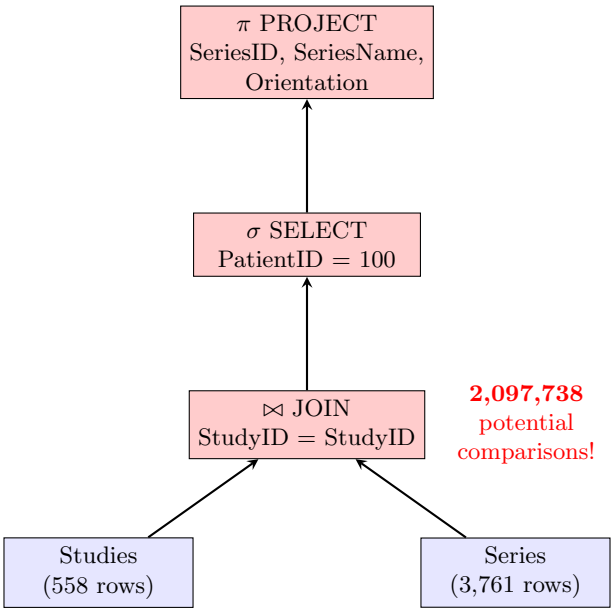


Figure 19: Initial "Canonical" Query Tree generated directly from the SQL parser (Inefficient Algebraic Order)

2. Transformation (Applying Heuristic Rules)
The optimizer applies specific algebraic rules to restructure the tree:

- **Rule 1: Push Down SELECT** ($\sigma_p(R \bowtie S) \rightarrow (\sigma_p(R)) \bowtie S$)**:** The filter `PatientID=100` is moved down, applying directly to the `Studies` relation. This reduces the input to the JOIN from 558 rows to just **2 rows**.

- **Rule 2: Push Down PROJECT** (π)**:** The projection of columns is pushed down to the `Series` relation, ensuring that only necessary columns are carried into memory for the JOIN.

3. Optimized State (Transformed Tree)
Figure 20 shows the final logical tree. By filtering early, the complexity of the JOIN operation is drastically reduced.

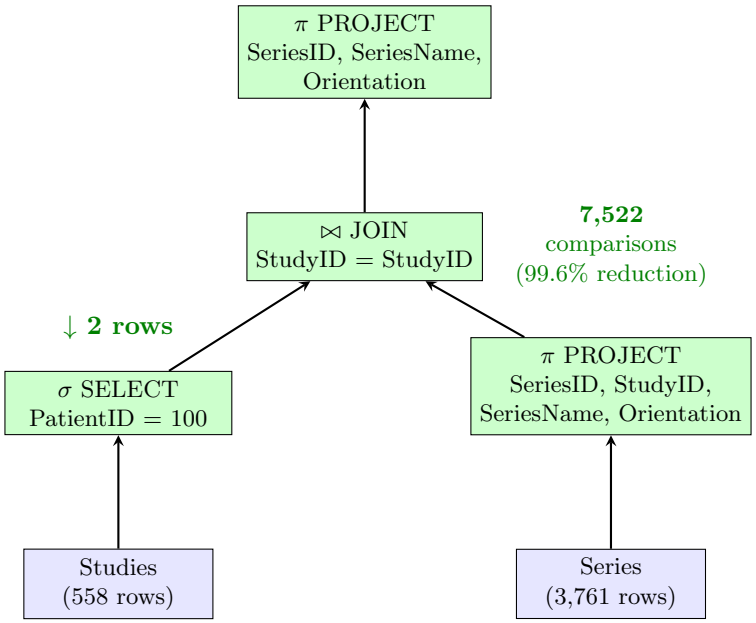


Figure 20: Transformed "Optimized" Query Tree after applying Heuristic Rules (Push-Down Selection/Projection)

Metric	Canonical Tree (Before)	Heuristic Tree (After)	Benefit
Operation Order	JOIN then FILTER	FILTER then JOIN	Early Reduction
Join Input (Left)	558 rows (Full Table)	**2 rows** (Filtered)	99.6% Reduction
Potential Comparisons	2,097,738	7,522	99.6% Reduction

Table 6: Impact of Heuristic Transformations on Logical Complexity

Note: This optimization happens purely at the logical level. The physical execution method (Scan vs. Seek) is determined in the next phase (Cost-Based Optimization).

5.2.2 Adding Physical Operators: From Logical to Physical Plan

After the logical tree is optimized by heuristics, the Query Optimizer enters Phase 3 (Cost-Based Optimization). It assigns specific physical algorithms (Scan vs. Seek, Hash vs. Nested Loops) to each logical operator based on estimated costs.

Query Tree 3: Physical Plan with Indexes (Case Study 2 Result)
Figure 21 shows how the logical JOIN is physically implemented after we added Foreign Key indexes.

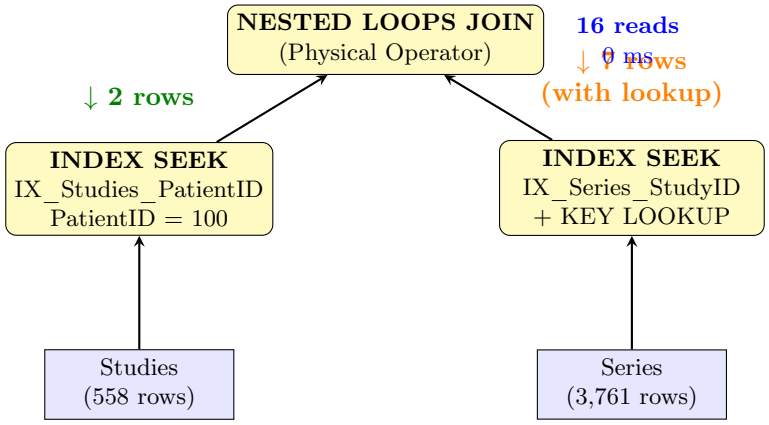


Figure 21: Physical Execution Plan with Index Seeks (Case Study 2 Result)

Optimization Decisions:

- **Heuristics:** Pushed down the filter, reducing `Studies` input to 2 rows.
- **Cost-Based:** Selected **Nested Loops** because the outer input (2 rows) is tiny.
- **Indexes:** Enabled **Index Seek**, avoiding full table scans.

Query Tree 4: Final Optimization with Covering Index (Case Study 3 Result)
Finally, Figure 22 shows the plan after creating a Covering Index. The Optimizer detects that the index contains all required columns, allowing it to remove the expensive Key Lookup.

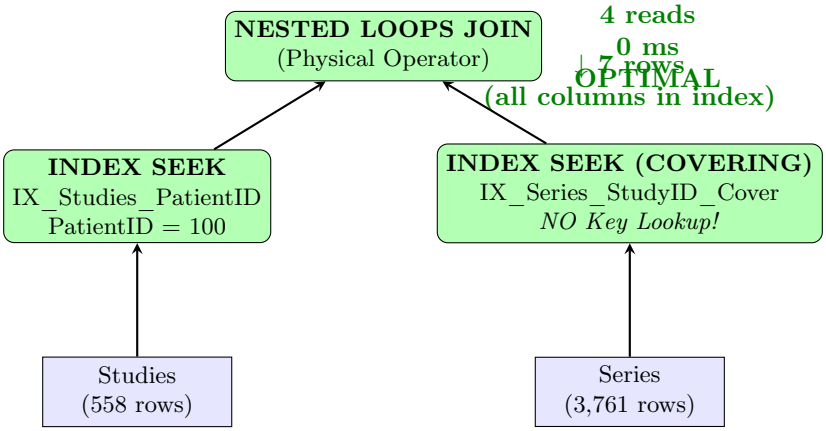


Figure 22: Final Optimized Plan with Covering Index (Case Study 3 Result)

Final Result: 4 reads, 0ms – fully optimized.

5.2.3 Summary: The Evolution of Query Trees (OLTP)

The synergy between logical heuristics and physical cost-based decisions is what enabled the massive performance gains in our OLTP case studies. Table 7 summarizes this optimization journey.

Tree Version	Description	Cost (Reads)
Tree 1 (Canonical)	Initial Parse – JOIN(558 × 3,761) then FILTER	Catastrophic
Tree 2 (Logical)	After Heuristics – FILTER(558 → 2) then JOIN	∼94 (est.)
Tree 3 (Physical)	After Indexes (CS2) – INDEX SEEK + NESTED LOOPS	16
Tree 4 (Optimal)	**Covering Index (CS3) – PURE SEEKS (No Lookups)**	**4**

Table 7: Evolution of Query Tree Optimization from Logical to Physical

Key Takeaway: Heuristics vs. Cost-Based Optimization
Our analysis confirms that optimization is a two-step partnership:

- **Heuristics (Logical Phase):** Prepared a "lean" query tree by pushing down filters, independent of data volume or indexes.
- **Cost-Based (Physical Phase):** Selected the fastest algorithms (Seeks, Nested Loops) based on the "lean" tree and available indexes.

Aspect	Heuristic Optimization	Cost-Based Optimization
When	Before cost calculation	After heuristic optimization
Goal	Reduce intermediate result sizes	Choose cheapest physical operators
Cost	Free (Algebraic rules)	Expensive (Statistics & Formulas)
Always applied	Yes	Depends on indexes, data distribution

Table 8: Comparison of Optimization Approaches

However, for massive analytical workloads, even the best B-Tree optimization has limits. This necessitates an architectural shift, which we explore in the next section.

5.3 Phase 4: Advanced Optimization (Architectural Shift)

5.3.1 Initial State (Canonical Tree)

Figure 23 illustrates the initial query tree generated by the parser. This represents the literal translation of the SQL query before any optimization rules are applied.

To understand the complexity of the initial tree, we first look at the SQL query and its translation into formal Relational Algebra.

1. Input SQL Query: The analytical query involves joining four tables and performing heavy aggregations.

```
SELECT p.PatientID,
       COUNT(DISTINCT st.StudyID), COUNT(DISTINCT ser.SeriesID),
       COUNT(i.ImageID), AVG(ser.FileCount)
FROM Patients p
JOIN Studies st ON p.PatientID = st.PatientID
JOIN Series ser ON st.StudyID = ser.StudyID
JOIN Images i ON ser.SeriesID = i.SeriesID
GROUP BY p.PatientID;
```

Implement 9: Case Study 4: Analytical Query

2. Canonical Relational Algebra Expression: The parser formally translates the `JOIN` clauses into a sequence of Cartesian Products ($\times$) followed by Selections (σ). The Aggregation (γ), Projection (π), and Sorting (τ) are applied sequentially on the result.

$$\text{Result} = \tau_{TotalImages\downarrow}\Big(\pi_{\substack{p.PatientID, TotalStudies,\\ TotalSeries, TotalImages, AvgFiles}}\Big(\gamma_{p.PatientID,\ \text{COUNT-DIST}(st.StudyID)\to TotalStudies,\ \text{COUNT-DIST}(ser.SeriesID)\to TotalSeries,\ \text{COUNT}(i.ImageID)\to TotalImages,\ \text{AVG}(ser.FileCount)\to AvgFiles}\Big(\sigma_{\substack{p.PatientID=st.PatientID\ \wedge\\ st.StudyID=ser.StudyID\ \wedge\\ ser.SeriesID=i.SeriesID}}\big((\text{Patients}\times\text{Studies})\times\text{Series}\big)\times\text{Images}\big)\Big)\Big)$$

3. Interpretation: This algebraic expression corresponds directly to the "Canonical Tree" shown in Figure 23.

- The innermost parentheses $((\text{Images}\times\text{Series})\times\dots)$ represent the creation of a massive intermediate dataset containing every possible combination of rows.
- The Selection σ (filtering matching IDs) is applied *after* these products are formed.
- The Aggregation γ and Projection π happen at the very end, meaning the system carries unnecessary data (like `FilePath`) through the entire pipeline.

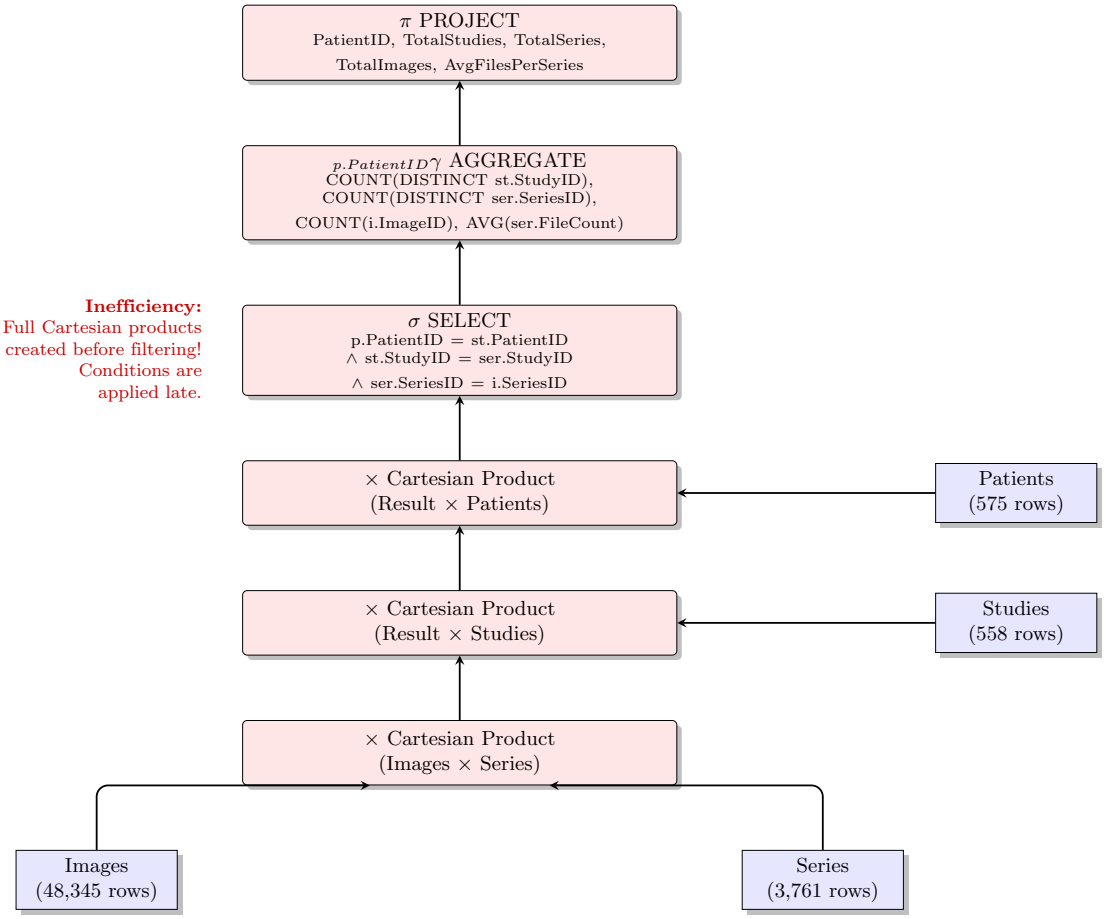


Figure 23: Initial "Canonical" Query Tree for Case Study 4 with Full Relational Algebra Expressions

5.3.2 Heuristic Optimization (Logical Transformation)

Before moving to physical execution, the optimizer applies heuristic rules to transform the inefficient "Canonical" tree into an Optimized Logical Tree.

The two most critical transformations for this analytical query are:

1. **Convert Cartesian Products to Joins ($\sigma\times \rightarrow\bowtie$):** The optimizer merges the Selection conditions (σ) with the Cartesian Products ($\times$) to form efficient Inner Joins ($\bowtie$).
2. **Push Down Projection (π):** Since the query does not filter rows (no `WHERE` clause), the most effective optimization is to reduce the width of the data. The optimizer injects `PROJECT` operators immediately after scanning the tables to discard unused columns (e.g., `FilePath`, `Notes`) before they enter the join pipeline.

Figure 24 visualizes this optimized logical structure.

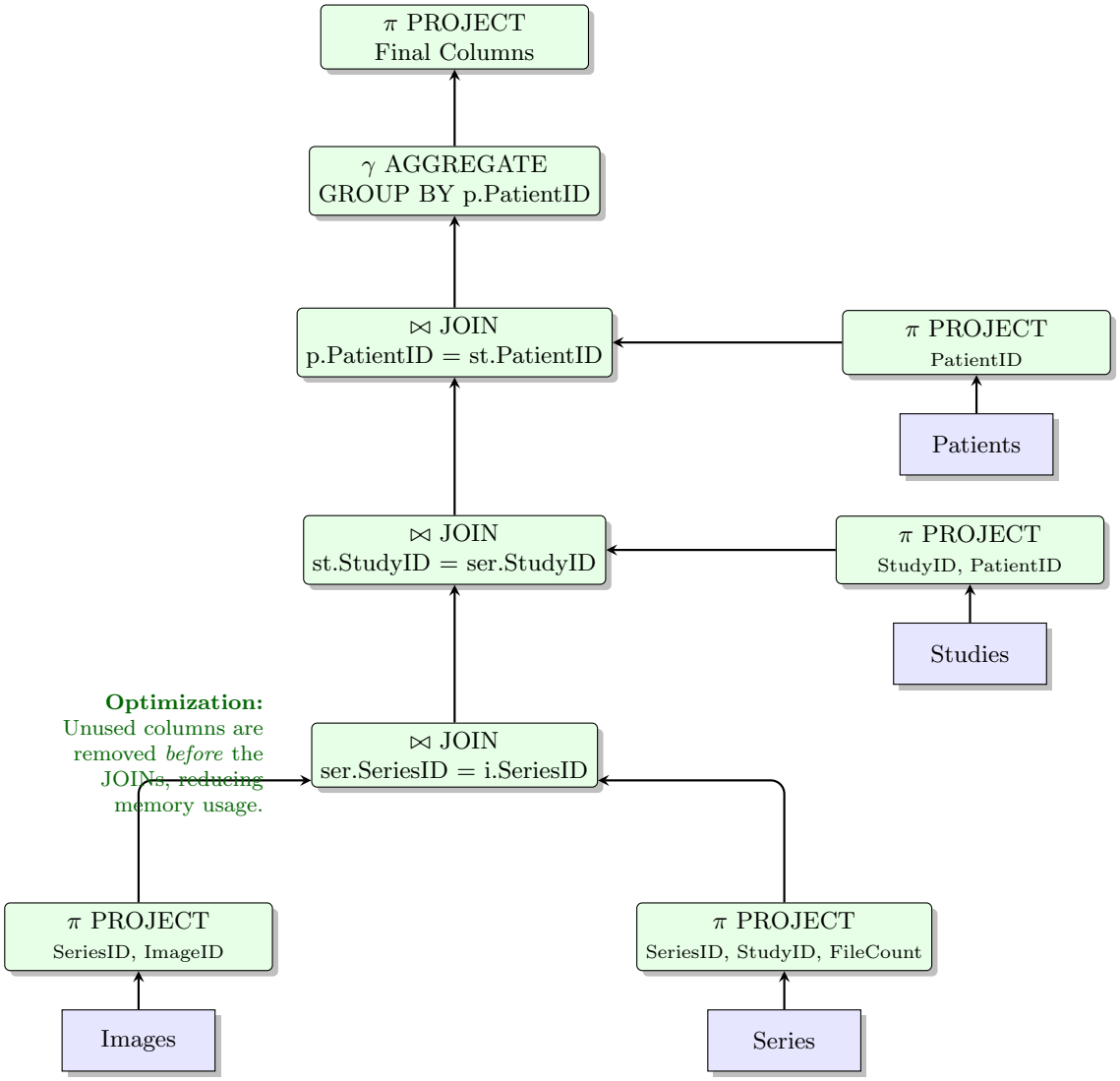


Figure 24: Optimized Logical Tree: Replacing Cartesian Products with Joins and Pushing Down Projections

This logical optimization prepares the ground for the physical execution. However, in a Row-Store architecture, "pushing down projection" only saves memory during processing; the engine still has to read the full 8KB pages from the disk. This limitation is exactly what the **Columnstore Index** (Section 5.4) overcomes physically.

5.3.3 From Logical Optimization to Physical Execution

While the heuristic transformations shown in Figure 24 successfully reduced the logical complexity (by pushing down projections), a critical physical limitation remains in the Row-Store architecture.

The "Row-Store" Bottleneck: Even though the logical plan only requests specific columns (e.g., `SeriesID`, `FileCount`), the physical storage engine stores data in 8KB pages containing *entire rows*. To read just one column, the engine must fetch the full page from the disk into memory.

- **Logical Gain:** Reduced memory usage during join processing.
- **Physical Pain:** I/O remains high because unused columns (like `FilePath`, `Notes`) are still physically read from the disk.

This physical limitation necessitates an architectural shift. To realize the full potential of the optimized logical tree, we need a storage format that aligns with the projection: **Columnstore Indexes**. This leads us to the final phase of physical optimization.

5.4 Analysis of QO Decision (Case Study 4: Row Mode vs. Batch Mode)

In Case Study 4 of Indexing, the creation of the Columnstore Index presented the Query Optimizer with a fundamental choice between two distinct execution architectures. This was not merely a choice of access path (like Scan vs. Seek), but a choice of **execution engine**.

5.4.1 The Decision Matrix

The Optimizer evaluated two competing paradigms based on estimated costs:

1. Paradigm A: Row-Store Execution (The "Before" Plan)

- **Mechanism:** The engine uses the standard Row Execution Mode. It reads data page-by-page (8KB) from the Clustered Index.
- **I/O Estimation:** High. To calculate `AVG(FileCount)`, the engine must read the entire row width, including unused columns like `FilePath` and `Note`, wasting memory bandwidth.
- **CPU & Memory Estimation:** High. The `Hash Match` operator processes rows one by one. Due to the complexity of `COUNT(DISTINCT)`, the memory grant is insufficient, forcing the engine to spill intermediate results to a **Table Spool** (Worktable).
- **Result:** Massive logical reads (98,656) and slow execution (357 ms).

2. Paradigm B: Column-Store Execution (The "After" Plan)

- **Mechanism:** The engine switches to **Batch Mode**, processing data in vectorized chunks (typically 900 rows per batch).
- **I/O Estimation:** Low. The `Columnstore Index Scan` reads only the specific compressed segments for `SeriesID` and `FileCount`, ignoring all other columns.
- **CPU & Memory Estimation:** Low. Batch Mode allows the `Hash Match` operator to aggregate data entirely in the CPU cache and RAM using SIMD (Single Instruction, Multiple Data) vector instructions.
- **Result:** The Table Spool is eliminated. I/O drops to near zero. Execution is instant (0-48 ms).

5.4.2 The Optimizer's Verdict

Comparing the estimated costs, the QO determined that Paradigm B (Batch Mode) offered an order-of-magnitude improvement over Paradigm A. It effectively abandoned the traditional row-based engine in favor of the vectorized processing model provided by the Columnstore Index.

5.5 Conclusion: Two Optimization Paradigms

This project has demonstrated that database optimization is not a one-size-fits-all process. As summarized in Table 9, different workloads require different optimization strategies.

Feature	**OLTP Paradigm (CS1-CS3)**	**OLAP Paradigm (CS4)**
Goal	Fast retrieval of specific rows	Fast aggregation of large datasets
Primary Tool	B-Tree Indexes (Seek)	Columnstore Indexes (Scan)
Execution Mode	**Row Mode** (Latency-optimized)	**Batch Mode** (Throughput-optimized)
Key Optimization	Reducing **Logical Reads** via Index Seeks	Reducing **CPU Cycles** via Vectorization

Table 9: Comparison of Optimization Paradigms: Row-Store vs. Column-Store

By understanding both the Logical phase (Heuristics/Tree Transformation) and the Physical phase (Cost-Based Selection/Architecture), a database administrator can effectively guide the system toward optimal performance for any given workload.

6 How DBMS works with transactions

This section presents a series of practical case studies performed on the `LumbarMRI` database to demonstrate how SQL Server manages transactions, ensures ACID properties, and handles concurrency through isolation levels.

All demonstrations are executed in Azure Data Studio / SQL Server Management Studio with explicit transaction control and appropriate diagnostic commands.

6.1 Case Study 1: Basic Transaction Control – COMMIT vs ROLLBACK

6.1.1 Scenario and Initial Analysis

Scenario: We insert a new patient and a related study in a single logical unit of work. We want to demonstrate that either both operations are permanently saved (`COMMIT`) or none of them are (`ROLLBACK`).

6.1.2 Query – Successful COMMIT

```
BEGIN TRANSACTION;

INSERT INTO Patient (PatientID)
VALUES (500);

INSERT INTO Studies (PatientID, StudyName, StudyDate)
VALUES (500, 'Lumbar MRI', GETDATE());

COMMIT TRANSACTION;

SELECT * FROM Patient WHERE PatientID = 600;
SELECT * FROM Studies WHERE StudyID = 600;
```

Implement 10: Case Study 1: Transaction with COMMIT

Result: Both rows are permanently persisted.

6.1.3 Query – ROLLBACK Scenario

```
BEGIN TRANSACTION;

INSERT INTO Patient (PatientID)
VALUES (600);

INSERT INTO Studies (PatientID, StudyName, StudyDate)
VALUES (600, 'Lumbar MRI', GETDATE());

-- Check current state (not yet commited)
SELECT * FROM Patient WHERE PatientID = 600;
SELECT * FROM Studies WHERE StudyID = 600;

-- Simulate an error or decision to cancel
ROLLBACK TRANSACTION;

SELECT * FROM Patient WHERE PatientID = 600;
SELECT * FROM Studies WHERE StudyID = 600;
```

Implement 11: Case Study 1: Transaction with ROLLBACK

Result: No rows are inserted – the database returns to its previous consistent state.

	PatientID
1	600

	StudyID	PatientID	StudyName	StudyDate
1	1563	600	Lumbar MRI	2025-11-13 09:49:31.847

	PatientID

	StudyID	PatientID	StudyName	StudyDate

Figure 25: No data after ROLLBACK – Atomicity demonstrated

6.1.4 Conclusion

Atomicity is fully enforced: partial changes are impossible.

Action	COMMIT	ROLLBACK
Data persisted?	Yes	No
Rows in Patient	1	0
Rows in Studies	1	0

Table 10: Atomicity – All or Nothing

6.2 Case Study 2: Error Handling with TRY...CATCH + Automatic ROLLBACK

6.2.1 Scenario

We intentionally provoke a primary-key violation inside a transaction and show how `TRY...CATCH` (SQL Server–specific) automatically rolls back the entire transaction when an error occurs.

6.2.2 Optimization Strategy: TRY...CATCH Block

```
BEGIN TRY
    BEGIN TRANSACTION;

    -- attempt insert with index violation
    INSERT INTO Patient (PatientID, PatientName) VALUES (1, 'DUPLICATE');
    INSERT INTO Studies (PatientID, StudyName, StudyDate)
    VALUES (600, 'FollowUp MRI', GETDATE());
```

```
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    PRINT 'Error detected: ' + ERROR_MESSAGE();
    ROLLBACK TRANSACTION;
END CATCH
```

Implement 12: Case Study 2: TRY...CATCH with automatic rollback

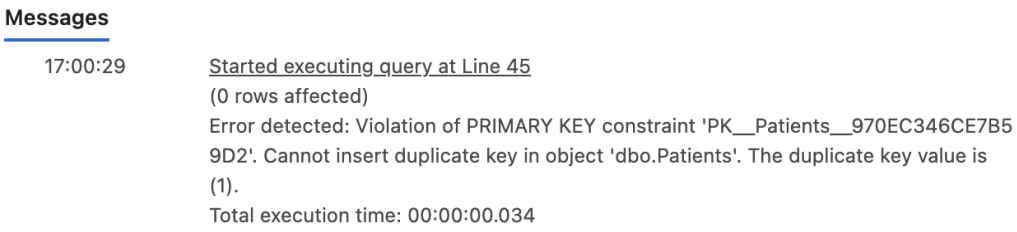


Figure 26: Error captured and transaction rolled back

6.2.3 Conclusion

SQL Server's `TRY...CATCH` guarantees that errors never leave the database in an inconsistent intermediate state.

Metric	Without TRY...CATCH	With TRY...CATCH
Transaction state after error	Partially committed (dangerous)	Fully rolled back
Developer control	None	Full (custom messages, logging)

Table 11: Error handling – Consistency preserved

6.3 Case Study 3: Concurrency – Dirty Read (READ UNCOMMITTED)

6.3.1 Scenario and Initial Analysis (Before Isolation)

Two sessions run concurrently:

- Session 1: starts a transaction, updates a row, waits 10 seconds, then `ROLLBACK`
- Session 2: reads the same row using `READ UNCOMMITTED`

6.3.2 Query – Session 1 (Modifier)

```
BEGIN TRANSACTION;
UPDATE dbo.ClinicalNotes SET CliniciansNote = 'Updated MRI' WHERE StudyID = 1;

-- simulate long running transaction
WAITFOR DELAY '00:00:10';
ROLLBACK TRANSACTION;
```

Implement 13: Session 1 – Creates dirty data

6.3.3 Query – Session 1-rcs (Modifier + Commit)

```
BEGIN TRANSACTION;
UPDATE dbo.ClinicalNotes SET CliniciansNote = 'Updated note by Session 1' WHERE NoteID = 1;

-- simulate short delay to overlap with Session 2
```

```
WAITFOR DELAY '00:00:5';
COMMIT TRANSACTION;
```

Implement 14: Session 1 – Creates dirty data

6.3.4 Query – Session 2 (Dirty Reader)

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
BEGIN TRANSACTION;
SELECT * FROM Studies WHERE StudyID = 566;
```

Implement 15: Session 2 – READ UNCOMMITTED

	StudyID	PatientID	StudyName	StudyDate
1	566	8	Updated MRI	2016-06-05 11:22:02.000

Figure 27: Dirty read – Session 2 sees data that will disappear

6.3.5 Conclusion

READ UNCOMMITTED sacrifices consistency for maximum concurrency.

Metric	READ COMMITTED (default)	READ UNCOMMITTED
Can see uncommitted data?	No	Yes (Dirty Read)
Blocking	Higher	Minimal
Consistency	Guaranteed	Sacrificed

Table 12: Dirty Read – Trade-off between consistency and performance

6.4 Case Study 4: Preventing Dirty Reads with READ_COMMITTED_SNAPSHOT

6.4.1 Optimization Strategy: Enable READ_COMMITTED_SNAPSHOT

```
ALTER DATABASE LumbarMRI SET READ_COMMITTED_SNAPSHOT ON;
```

Implement 16: Activate RCS at database level

6.4.2 Query – Session 2-rcs (See only committed values)

```
BEGIN TRANSACTION;

-- first select which sees the original version
SELECT * FROM dbo.ClinicalNotes SET WHERE StudyID = 1;

-- simulate long running transaction
WAITFOR DELAY '00:00:6';

-- sees the committed version
SELECT * FROM dbo.ClinicalNotes SET WHERE StudyID = 1;

COMMIT TRANSACTION;
```

Implement 17: Session 1 – Creates dirty data

6.4.3 Results and Verification (After Activation)

Same scenario as Case Study 3, but Session 2 now automatically sees only committed data, even under the default `READ COMMITTED` isolation level.

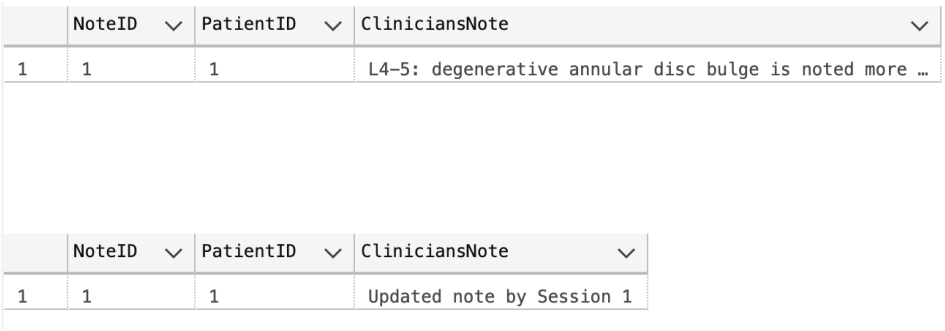


Figure 28: With RCS ON – no dirty read, no blocking

6.4.4 Conclusion

`READ_COMMITTED_SNAPSHOT` is the recommended setting for most OLTP workloads on SQL Server (zero dirty reads + almost no read blocking).

Metric	Locking R.C.	RCS ON	Improvement
Dirty reads	Prevented	Prevented	Same
Reader blocking	Yes	Almost none	Huge
Writer blocking readers	Yes	No	Eliminated
tempdb usage	None	Moderate	Trade-off

Table 13: READ_COMMITTED_SNAPSHOT – Best of both worlds

6.5 Case Study 5: Full Transaction Consistency with SNAPSHOT Isolation

6.5.1 Optimization Strategy: Enable and Use SNAPSHOT

```
ALTER DATABASE DoctorNotes SET ALLOW_SNAPSHOT_ISOLATION ON;
ALTER DATABASE DoctorNotes SET READ_COMMITTED_SNAPSHOT ON;

USE DoctorNotes; GO
```

Implement 18: Enable SNAPSHOT isolation

6.5.2 Query – Reading Session under SNAPSHOT

```
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
BEGIN TRANSACTION;

-- sees original version
SELECT * FROM dbo.ClinicalNotes WHERE CliniciansNote = 1;

-- Session 1 commit
WAITFOR DELAY '00:00:6';

-- still sees original
SELECT * FROM dbo.ClinicalNotes WHERE CliniciansNote = 1;
COMMIT;
```

Implement 19: Session 2 – SNAPSHOT isolation

	NoteID	PatientID	CliniciansNote
1	1	1	L4-5: degenerative annular disc bulge is noted more …

	NoteID	PatientID	CliniciansNote
1	1	1	L4-5: degenerative annular disc bulge is noted more …

Figure 29: Both SELECTs return identical data – transaction-level consistency

6.5.3 Conclusion

SNAPSHOT isolation provides the strongest repeatable-read guarantee without any locking contention.

Isolation Level	Dirty Read	Non-repeatable Read	Reader/Writer Blocking
READ UNCOMMITTED	Yes	Yes	None
READ COMMITTED (locking)	No	Yes	High
READ_COMMITTED_SNAPSHOT	No	Yes	Very Low
SNAPSHOT	No	No	None

Table 14: Comparison of isolation levels in SQL Server

6.6 Case Study 6: Transaction Monitoring Tools (SQL Server Specific)

6.6.1 Scenario and Initial Analysis

In real production environments, it is essential to be able to observe active transactions in real time: which session is blocking, how long a transaction has been open, what command is currently running, etc. SQL Server offers powerful built-in tools — **Dynamic Management Views (DMVs)** — that are completely exclusive to the Microsoft engine.

6.6.2 Query – Real-time Monitoring with DMVs

```
SELECT
    at.transaction_id,
    at.name AS transaction_name,
    at.transaction_state,
    at.transaction_type,
    s.session_id,
    s.host_name,
    s.login_name,
    r.status AS request_status,
    r.command
FROM sys.dm_tran_active_transactions at
JOIN sys.dm_tran_session_transactions st
     ON at.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s
     ON st.session_id = s.session_id
LEFT JOIN sys.dm_exec_requests r
```

```
    ON s.session_id = r.session_id;
```

Implement 20: Case Study 6: Complete query to monitor active transactions

6.6.3 Results and Verification

When one or more transactions are active (for example, an open `BEGIN TRANSACTION` without `COMMIT/ROLLBACK`), the query instantly returns all relevant information.

6.6.4 Analysis of Key Columns

- `transaction_id` / `name`: unique identifier and explicit name (if assigned with `BEGIN TRAN MyTrans`)
- `transaction_state`: 2 = active (most common)
- `session_id`, `login_name`, `host_name`: who is responsible
- `command`: often `WAITFOR`, `SELECT`, `UPDATE`... very useful for finding long-running queries
- `request_status`: `runnable`, `suspended`, `running`

6.6.5 Conclusion

Thanks to DMVs, a DBA can detect in a fraction of a second any problematic transaction without using external tools.

Information	**DMV used**	**Practical use**
Active transactions	sys.dm_tran_active_transactions	See open transactions
Sessions linked to transactions	sys.dm_tran_session_transactions	Find the responsible user
Session details (login, machine)	sys.dm_exec_sessions	Identify the source
Current command and status	sys.dm_exec_requests	Detect blocks / long queries

Table 15: Summary of DMVs used for transaction monitoring

This real-time visibility is a major advantage of SQL Server over many other DBMS that require external profilers or paid tools to obtain the same information.

7 How DBMS works with concurrency control

Concurrency control is essential in a multi-user environment to ensure the Isolation property of ACID and preserve database consistency. SQL Server uses locking mechanisms to manage concurrent access to resources. In this section, we demonstrate how the Lock Manager handles conflicts using the LumbarMRI database.

7.1 Case Study 1: Blocking (Lock Contention)

7.1.1 Scenario and Initial Analysis

In this scenario, we simulate a standard Read-Write conflict.

- **Session 1 (Writer):** Updates a specific study record (StudyID = 1) within a transaction but does not immediately commit. This transaction holds an Exclusive Lock (X) on the row.
- **Session 2 (Reader):** Attempts to read the same study record. This operation requires a Shared Lock (S).

According to the Two-Phase Locking (2PL) protocol compatibility matrix, an Exclusive Lock (X) is not compatible with a Shared Lock (S). Therefore, Session 2 must wait (block) until Session 1 releases the lock.

7.1.2 Implementation Steps

Step 1: Session 1 (The Blocker)

```
USE LumbarMRI;
GO

PRINT '========================================';
PRINT 'SESSION 1: Begin update transaction';
PRINT '========================================';
PRINT '';

BEGIN TRANSACTION;

    PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 1: Updating StudyID = 1...';

    UPDATE Studies
    SET StudyName = 'Blocking Test - Session 1'
    WHERE StudyID = 1;

    PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 1: Acquired EXCLUSIVE LOCK';
    PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 1: Holding lock for 15 seconds...';
    PRINT '';
    PRINT '>>> NOW RUN SESSION 2 IMMEDIATELY! <<<';
    PRINT '';

    WAITFOR DELAY '00:00:15';

    PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 1: Rolling back...';

ROLLBACK TRANSACTION;

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 1: Lock released';
PRINT 'SESSION 2 is now allowed to proceed!';
GO
```

Implement 21: Case Study 1: Session 1 - Blocking

We execute an update transaction that holds the lock for 15 seconds.

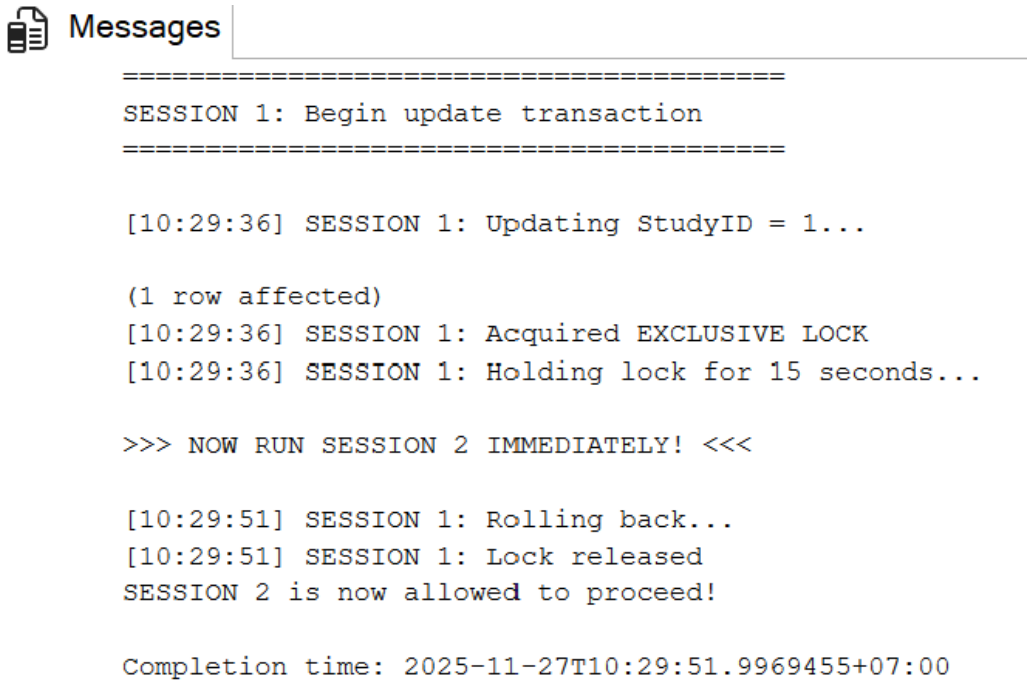


Figure 30: Session 1 acquires an Exclusive Lock (X) on Study with StudyID = 1

Step 2: Session 2 (The Blocked)

```
USE LumbarMRI;
GO

PRINT '========================================';
PRINT 'SESSION 2: Attempting to read data';
PRINT '========================================';
PRINT '';

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 2: Requesting SHARED LOCK...';
PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 2: Waiting... (BLOCKED)';
PRINT '';

SELECT
    StudyID,
    PatientID,
    StudyName,
    StudyDate
FROM Studies
WHERE StudyID = 1;

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 2: Read successful!';
PRINT 'SESSION 2 was BLOCKED for approximately 15 seconds';
GO
```

Implement 22: Case study 1: Session 2 - Blocking

While Session 1 is running, we execute a SELECT statement in a separate window.

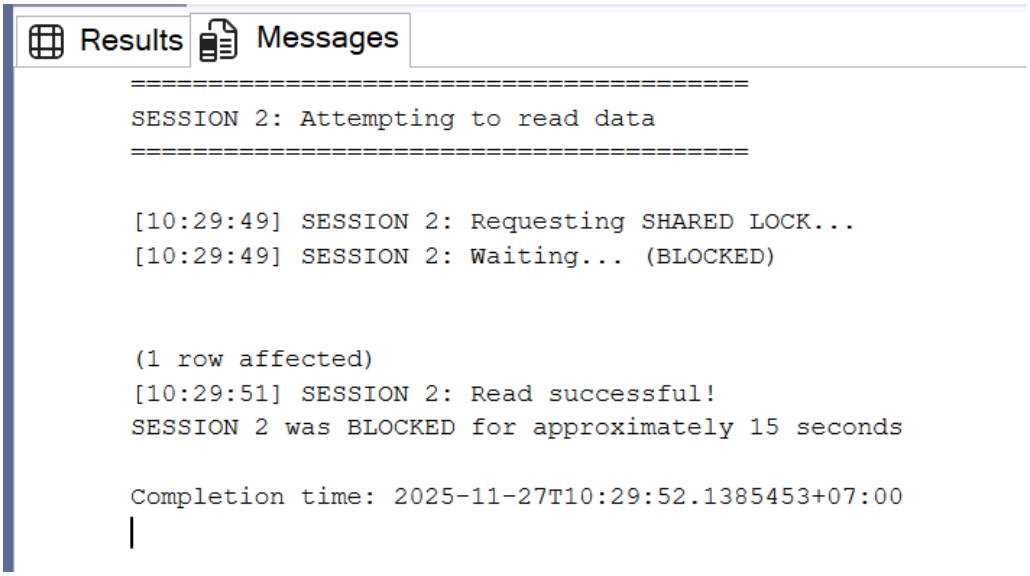


Figure 31: Session 2 is blocked, waiting for the Exclusive Lock to be released

7.1.3 Results and Verification

The observation confirms the strict concurrency control mechanism:

- Session 2 did not return data immediately. It entered a **WAIT** state.
- Once Session 1 completed (after 15 seconds) or issued a `ROLLBACK`, Session 2 immediately returned the result.

7.1.4 Conclusion

This case study demonstrates the **Pessimistic Locking** model used by SQL Server by default. To enforce Isolation, the DBMS prevents "Dirty Reads" by forcing readers to wait for writers to commit, ensuring that Session 2 never reads uncommitted (potentially inconsistent) data.

7.2 Case Study 2: Deadlock Simulation

7.2.1 Scenario and Initial Analysis

A deadlock is a situation where two or more transactions are permanently blocked because each transaction holds a lock on a resource that the other transaction needs. This creates a cycle in the **Wait-For Graph**.

In this scenario, we simulate a classic cyclic dependency:

- **Session 1:** Acquires an Exclusive Lock (X) on the *Studies* table, then attempts to update the *Series* table.
- **Session 2:** Acquires an Exclusive Lock (X) on the *Series* table, then attempts to update the *Studies* table.

Without DBMS intervention, both sessions would wait indefinitely.

7.2.2 Implementation Steps

```
USE LumbarMRI;
GO
```

```
PRINT '================================================================';
PRINT 'SESSION 1 (Transaction A): Starting';
PRINT '================================================================';
PRINT '';

SET DEADLOCK_PRIORITY LOW;

BEGIN TRANSACTION;

PRINT'['+CONVERT(VARCHAR, GETDATE(),108)+']Step1: Acquiring X Lock on Studies(StudyID=1)';

    UPDATE Studies
    SET StudyName = 'Session A - Step 1'
    WHERE StudyID = 1;

    PRINT'['+CONVERT(VARCHAR, GETDATE(), 108)+']SESSION1: Holding X Lock on Studies';
    PRINT'';
    PRINT'>>>NOW RUN SESSION2 IMMEDIATELY!<<<';
    PRINT'';
PRINT'['+CONVERT(VARCHAR, GETDATE(),108)+']SESSION1: Waiting 7 seconds for SESSION 2 to acquire lock on Series.

    WAITFOR DELAY '00:00:07';

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Step 2 - Attempting to acquire X Lock on Series (SeriesID=1)'
PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Series is locked by SESSION 2...';
PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Waiting for SESSION 2 to release Series...';
    PRINT '';
    PRINT '>>> DEADLOCK WILL OCCUR! <<<';
    PRINT '';

    UPDATE Series
    SET SeriesName = 'Session A - Step 2'
    WHERE SeriesID = 1;

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Successfully acquired lock on Series';
PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SUCCESS - SESSION 2 was chosen as Deadlock Victim';

COMMIT TRANSACTION;

PRINT '';
PRINT '================================================================';
PRINT 'SESSION 1: Completed successfully';
PRINT 'SESSION 2 was ROLLED BACK (Deadlock Victim)';
PRINT '================================================================';
GO
```

Implement 23: Case study 2: Session 1 - Deadlock Detection

Step 1: Execute Session 1 Execute Session 1 In the first query window, we start a transaction that locks Studies and waits before accessing Series.

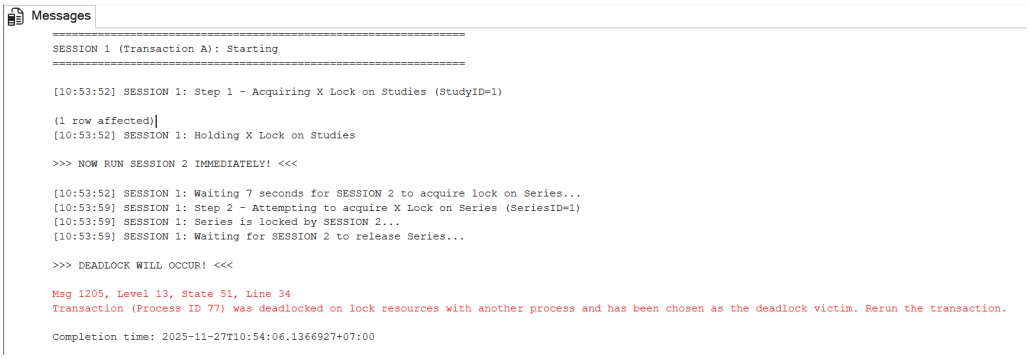


Figure 32: Session 1 acquires lock on Studies and requests lock on Series

Step 2: Execute Session 2 Immediately in a second window, we start a transaction that locks *Series* and tries to access *Studies*.

```
USE LumbarMRI;
GO

PRINT '================================================================';
PRINT 'SESSION 2 (Transaction B): Starting';
PRINT '================================================================';
PRINT '';

SET DEADLOCK_PRIORITY HIGH;

BEGIN TRANSACTION;

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Step 1 - Acquiring X Lock on Series (SeriesID=1)';

    UPDATE Series
    SET SeriesName = 'Session B - Step 1'
    WHERE SeriesID = 1;

    PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 2: Holding X Lock on Series';
    PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SESSION 2: Waiting 7 seconds...';
    PRINT '';

    WAITFOR DELAY '00:00:07';

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Step 2 - Attempting to acquire X Lock on Studies (StudyID=1)';
PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Studies is locked by SESSION 1...';
PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Waiting for SESSION 1 to release Studies...';
    PRINT '';
    PRINT '>>> WAIT-FOR GRAPH: SESSION 1 to SESSION 2 to SESSION 1 (CYCLE!) <<<';
    PRINT '';

    UPDATE Studies
    SET StudyName = 'Session B - Step 2'
    WHERE StudyID = 1;

PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] Successfully acquired lock on Studies';
PRINT '[' + CONVERT(VARCHAR, GETDATE(), 108) + '] SUCCESS - SESSION 1 was chosen as Deadlock Victim';

COMMIT TRANSACTION;

PRINT '';
PRINT '================================================================';
```

```
PRINT 'SESSION 2: Completed successfully';
PRINT 'SESSION 1 was ROLLED BACK (Deadlock Victim)';
PRINT '============================================================';
GO
```

Implement 24: Case study 2: Session 2 - Deadlock Detection

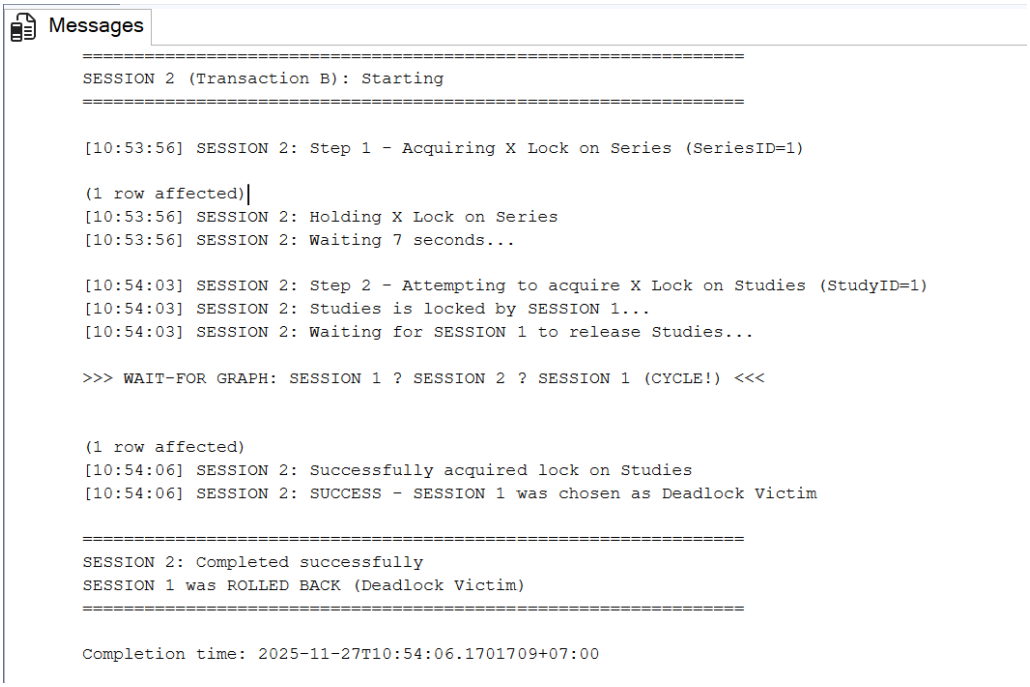


Figure 33: Session 2 acquires lock on Series and requests lock on Studies

7.2.3 Results and Verification

After a few seconds, the SQL Server Lock Monitor detects the cycle. It arbitrarily chooses one transaction as the "deadlock victim" and terminates it to allow the other to proceed.

- **Victim Session:** Returns Error 1205: *"Transaction (Process ID X) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction."*
- **Survivor Session:** Completes successfully.

7.2.4 Conclusion

This case study demonstrates the active role of the DBMS in **Deadlock Detection and Resolution**. By maintaining a **Wait-For Graph** and periodically checking for cycles, the system ensures that resource contention does not freeze the database indefinitely, preserving availability and consistency.

The victim selection is based on:

- `DEADLOCK_PRIORITY`: Prioritizing transactions based on assigned levels (e.g., Session 1 set to LOW vs Session 2 set to HIGH). In this scenario, the DBMS honors the priority setting and selects Session 1 as the victim.
- **Transaction cost**: In the absence of priority settings (or if priorities are equal), smaller transactions (those that have generated fewer log records) are preferred as victims to minimize the rollback overhead.

This automatic deadlock resolution mechanism is critical for maintaining system availability in multi-user database environments.

8 Comparisons with another DBMS

8.1 Importing a 6GB MRI Dataset into MongoDB

The dataset is organized hierarchically on disk:

- Patient folder
- Each patient contains multiple studies
- Each study contains multiple series
- Each series contains a sequence of MRI slice files (`.ima`)

To preserve this hierarchical structure, the dataset is imported into MongoDB using Python and the `pymongo` driver. Each patient becomes a document in the `patients` collection, containing nested subdocuments for studies, series, and lists of images.

8.1.1 Python Import Script

The Python script below recursively scans all patient, study, and series folders and inserts the data structure into MongoDB as nested JSON documents.

```
import os
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["mri_db"]
col = db["mri_images"]
def import_mri_folder(root_folder):
    for patient in os.listdir(root_folder):
        patient_path = os.path.join(root_folder, patient)
        if not os.path.isdir(patient_path):
            continue
        for study in os.listdir(patient_path):
            study_path = os.path.join(patient_path, study)
            if not os.path.isdir(study_path):
                continue
            for series in os.listdir(study_path):
                series_path = os.path.join(study_path, series)
                if not os.path.isdir(series_path):
                    continue
                for image_folder in os.listdir(series_path):
                    img_folder_path = os.path.join(series_path, image_folder)
                    if not os.path.isdir(img_folder_path):
                        continue
                    for file in os.listdir(img_folder_path):
                        file_path = os.path.join(img_folder_path, file)
                        if os.path.isfile(file_path):
                            doc = {
                                "patientID": patient,
                                "studyID": study,
                                "seriesID": series,
                                "imageFolder": image_folder,
                                "fileName": file,
                                "path": file_path.replace("\\", "/")
                            }
                            col.insert_one(doc)


import_mri_folder("D:/MRI_Data")
```

8.1.2 Document Structure in MongoDB

A sample document in the `patients` collection after import:

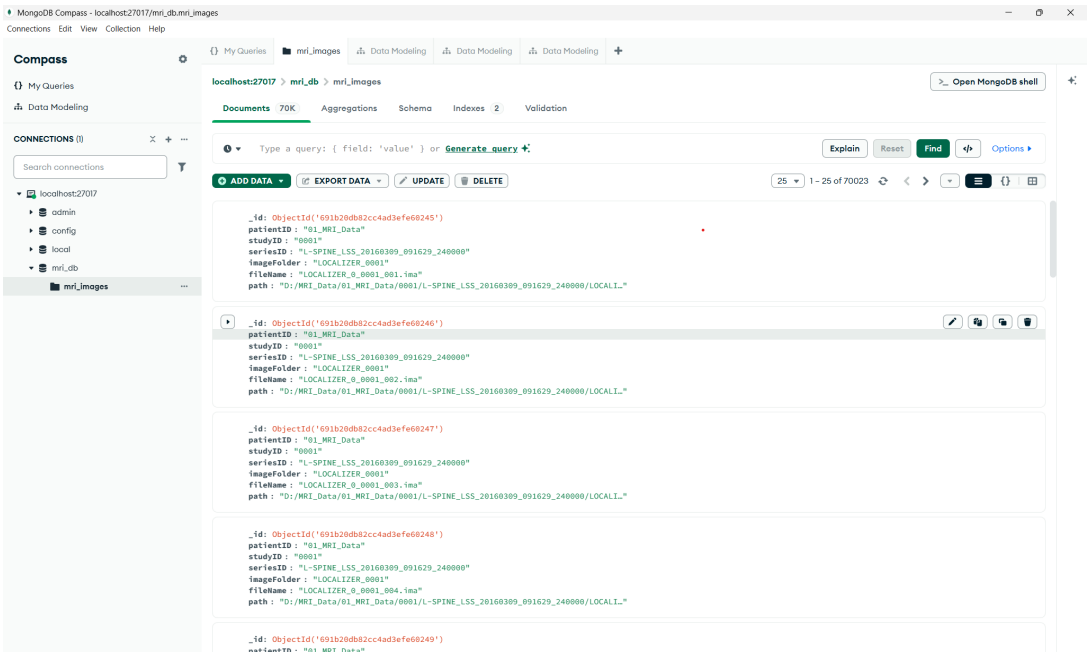


Figure 34: Data in MongoDB Compass

8.2 Case Study 1: Optimizing a Highly Selective `find()` Query

To demonstrate how MongoDB leverages B-tree indexing to optimize query performance, this case study performs a controlled experiment on the `mri_images` collection. We analyze performance **before** and **after** creating an index, using the command:

```
db.mri_images.find({ fileName: "T1_TSE_SAG__0005_013.ima" })
```

MongoDB performance metrics were captured using:

- **db.collection.explain("executionStats")**: shows the query plan
- **executionTimeMillis**: total execution time
- **totalDocsExamined**: number of scanned documents
- **totalKeysExamined**: number of scanned index keys

8.2.1 Scenario and Initial Analysis (Before Optimization)

Scenario: The query searches for a single MRI slice by exact filename. This is an extremely selective filter returning only **2 matching documents** out of **70,023 total entries**. Before an index is created, MongoDB has no choice but to perform a full collection scan.

Query:

```
db.mri_images.find({
    fileName: "T1_TSE_SAG__0005_013.ima"
})
```

Implement 25: MongoDB Query (Unoptimized)

Execution Plan:
MongoDB returned the following plan using `explain("executionStats")`:


```
{
  "explainVersion": "1",
  "queryPlanner": {
    "namespace": "mri_db.mri_images",
    "parsedQuery": { "fileName": { "$eq": "T1_TSE_SAG__0005_013.ima" } },
    "winningPlan":{"stage":"COLLSCAN","filter":{"fileName":{"$eq":"T1_TSE_SAG__0005_013.ima"}}}
  },
  "executionStats": {
    "executionSuccess": true,
    "nReturned": 2,
    "executionTimeMillis": 23,
    "totalKeysExamined": 0,
    "totalDocsExamined": 70023,
    "executionStages": {
      "stage": "COLLSCAN",
      "docsExamined": 70023,
      "nReturned": 2,
      "executionTimeMillisEstimate": 21
    }
  }
}

```


Implement 26: Execution Plan Before Indexing (COLLSCAN) in MongoDB

Initial Metrics:

- **Stage:** COLLSCAN (full collection scan)
- **Documents Scanned:** 70,023
- **Index Keys Scanned:** 0
- **Returned Documents:** 2
- **Execution Time:** 23 ms

The unindexed query requires scanning the entire dataset, which grows linearly with data size.

8.2.2 Optimization Strategy: Creating a B-tree Index

Rationale:
Since `fileName` is highly selective and frequently queried, it is an ideal candidate for indexing. MongoDB uses a **B-tree index** to perform logarithmic-time lookups instead of scanning all 70,023 documents.

Index Creation:

```
db.mri_images.createIndex({ fileName: 1 })
```

Implement 27: Create Index on fileName

This constructs a B-tree index where each leaf node stores pointers to matching documents.

8.2.3 Results and Verification (After Optimization)

Query: We run the same query again:

```
db.mri_images.find({
    fileName: "T1_TSE_SAG__0005_013.ima"
})
```

Execution Plan (Indexed):

```
{
  "explainVersion": "1",
  "queryPlanner": {
    "namespace": "mri_db.mri_images",
    "parsedQuery": { "fileName": { "$eq": "T1_TSE_SAG__0005_013.ima" } },
    "winningPlan": {
      "stage": "FETCH",
      "inputStage": {
        "stage": "IXSCAN",
        "keyPattern": { "fileName": 1 },
        "indexName": "fileName_1",
    "indexBounds":{"fileName":["[\"T1_TSE_SAG__0005_013.ima\",\"T1_TSE_SAG__0005_013.ima\"]"]}
      }
    }
  },
  "executionStats": {
    "executionSuccess": true,
    "nReturned": 2,
    "executionTimeMillis": 4,
    "totalKeysExamined": 2,
    "totalDocsExamined": 2,
    "executionStages": {
      "stage": "FETCH",
      "docsExamined": 2,
      "nReturned": 2,
      "inputStage": {
        "stage": "IXSCAN",
        "keysExamined": 2,
        "indexName": "fileName_1"
      }
    }
  }
}

```

Implement 28: MongoDB Execution Plan After Indexing (IXSCAN)

Performance Metrics After Indexing:

- **Stage:** IXSCAN (index scan) + FETCH
- **Index Keys Scanned:** 2
- **Documents Scanned:** 2
- **Returned Documents:** 2
- **Execution Time:** 4 ms

This represents a dramatic performance improvement.

8.2.4 Summary and Comparison

Metric	Before (No Index)	After (With Index)	Improvement
Operator	COLLSCAN	IXSCAN + FETCH	Major Improvement
Documents Scanned	70,023	2	**99.997% Reduction**
Index Keys Scanned	0	2	Uses B-tree
Execution Time	23 ms	4 ms	**82.6% Faster**

Table 16: Performance Comparison for MongoDB Case Study 1

8.2.5 Interpretation

Without an index:

$$\text{Time Complexity} = O(n)$$

With a B-tree index:

$$\text{Time Complexity} = O(\log n)$$

As dataset size increases, the performance gap becomes exponentially more significant.

8.3 Case Study 2: Emulating Joins in MongoDB with Aggregation`find()` Query

8.3.1 MongoDB with Aggregation

Scenario: We aim to aggregate MRI image file names per study and series for a specific patient ID (`"01_MRI_Data"`). The collection `mri_images` contains all hierarchical data in a single collection. The initial goal is to compare query performance before and after creating a simple index on `patientID`.

Query Pipeline: The aggregation pipeline emulates the hierarchical JOINs used in the SQL environment:

```
db.mri_images.aggregate([
    {
        $match: { patientID: "01_MRI_Data" }
    },
    {
        $group: {
            _id: { studyID: "$studyID", seriesID: "$seriesID" },
            images: { $push: "$fileName" }
        }
    },
    {
        $project: {
            _id: 0,
            studyID: "$_id.studyID",
            seriesID: "$_id.seriesID",
            images: 1
        }
    }
]);
```

Before Indexing (Baseline)

- Stage: `COLLSCAN` → `GROUP` → `PROJECT`
- Documents Examined: 70,023
- Documents Returned: 558

- Execution Time: 116 ms

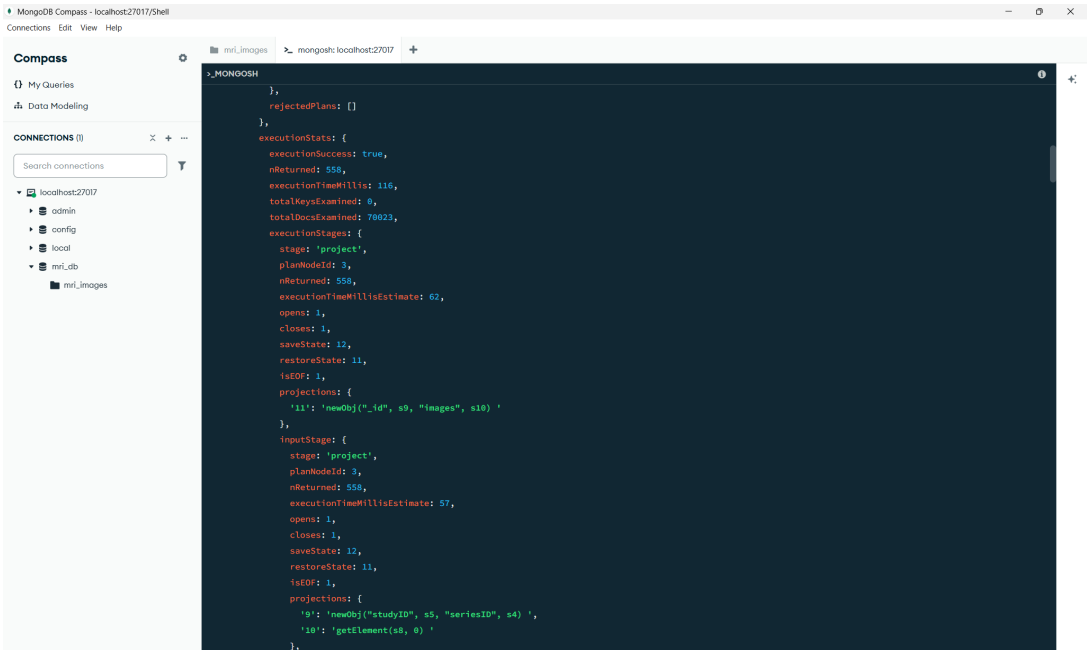


Figure 35: Before indexing in MongoDB Compass

```
db.mri_images.aggregate([
    { $match: { patientID: "01_MRI_Data" } }
]).explain("executionStats")
```

After Simple Indexing (Index on `patientID`)

- Index: `db.mri_images.createIndex({ patientID: 1 })`
- Stage: `IXSCAN` → `FETCH` → `GROUP` → `PROJECT`
- Keys Examined: 70,023
- Documents Examined: 70,023
- Documents Returned: 558
- Execution Time: 200 ms

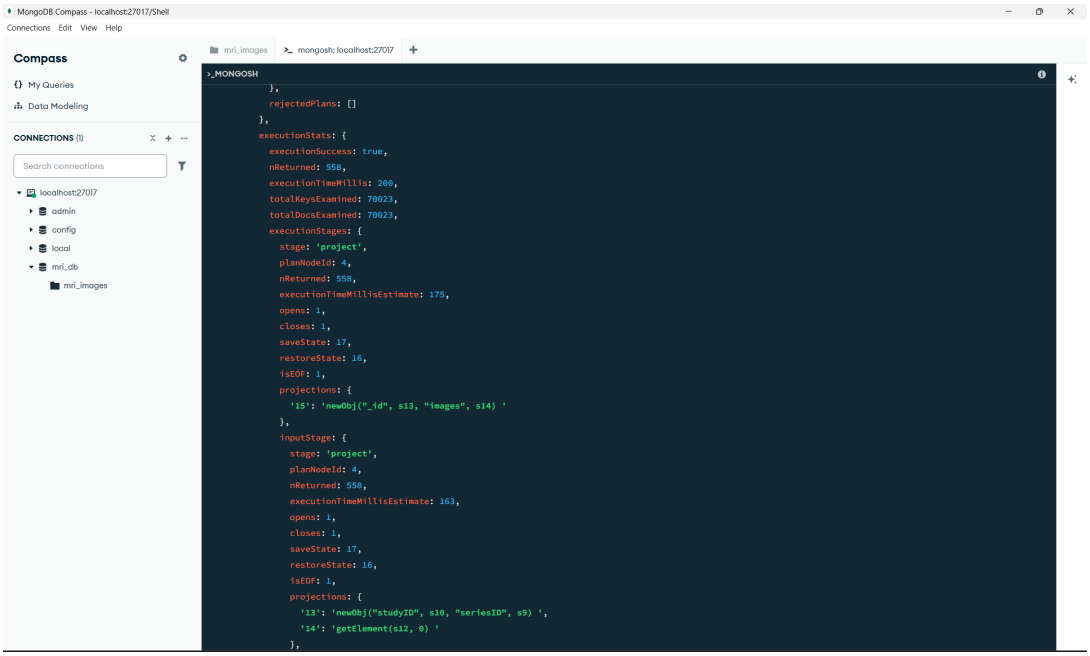


Figure 36: Before indexing in MongoDB Compass

Case Study 2 Analysis

- Indexing `patientID` enables MongoDB to use `IXSCAN`, similar to indexing foreign keys in SQL, reducing scanning cost for top-level filter.
- Aggregation (*group*+*push*) still examines all matching documents, so execution time is dominated by memory/CPU overhead.
- Conceptually, this mirrors SQL Case Study 2: index improves the filtering/seek, but join or aggregation of large subdocuments may remain a bottleneck.

Key Takeaways

- Indexing top-level field (`patientID`) is critical for performance when filtering.
- Aggregation operations may still be expensive if many subdocuments are pushed into arrays.

8.3.2 Eliminating FETCH with a Covering Index in MongoDB

Scenario: In Case Study 2, using an index on `patientID` improved the initial *match* filtering. However, the aggregation pipeline still required MongoDB to **fetch all 70,023 matching documents** to perform the *group* and *push* operations. This created a new bottleneck analogous to SQL's Key Lookup.

Analysis of "Before" State:

- **Index Used:** `patientID_1` (single-field index)
- **Documents Examined:** 70,023
- **Execution Time:** 200 ms
- **Observation:** Index improves the initial *match* stage, but *group* and *push* stages still require reading document data (FETCH), which consumes significant memory and CPU.

Optimization Strategy: Create a Covering Index To eliminate the FETCH stage, we can create a **compound index covering all fields used in the aggregation pipeline**:

```
db.mri_images.createIndex(
    { patientID: 1, studyID: 1, seriesID: 1, fileName: 1 }
);
```

Implement 29: MongoDB: Creating a Covering Index

- Now, the *match*, *group*, and *project* stages can be satisfied directly from the index. - This is equivalent to SQL Server's `Covering Index` that eliminates Key Lookups.

Results and Verification (After Covering Index)

Aggregation Query (same as before):

```
db.mri_images.aggregate([
  { $match: { patientID: "01_MRI_Data" } },
  { $group: {
      _id: { studyID: "$studyID", seriesID: "$seriesID" },
      images: { $push: "$fileName" }
  }},
  { $project: {
      _id: 0,
      studyID: "$_id.studyID",
      seriesID: "$_id.seriesID",
      images: 1
  }}
]);
```

Implement 30: MongoDB Aggregation Query

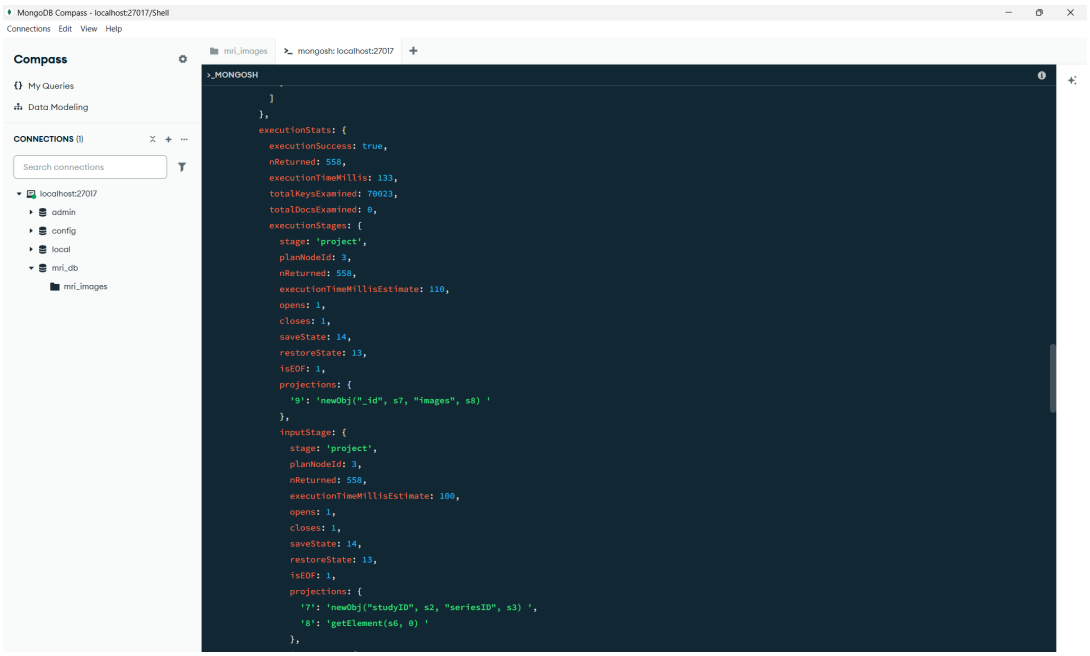


Figure 37: Covering index in MongoDB Compass

Analysis of Optimized State:

- **Index Used:** `patientID_1_studyID_1_seriesID_1_fileName_1` (covering index)
- **Keys Examined:** 70,023
- **Documents Examined:** 0 (no FETCH stage, only index keys read)

- **Execution Time:** 133 ms
- **Observation:** The `FETCH` stage is eliminated; MongoDB retrieves data directly from the index, reducing I/O and CPU usage.

Metric	Before Covering Index	After Covering Index	Improvement
Primary Bottleneck	FETCH documents	(None)	Eliminated
Documents Examined	70,023	Index keys only	Reduced
Execution Time	200 ms	133 ms	33% faster

Table 17: Comparison of Performance Metrics for MongoDB Case Study 3

Conclusion:
By creating a covering index that includes all fields needed by the aggregation, we eliminated the FETCH bottleneck, similar to SQL Server's Covering Index eliminating Key Lookups. This demonstrates that **NoSQL databases can benefit from the same index-covering optimization principles**.

1. **CS1:** *match* with index reduces full collection scan.
2. **CS2:** Index on foreign keys (patientID) reduces aggregation input.
3. **CS3:** Covering index eliminates FETCH, analogous to Key Lookup elimination in SQL.

8.4 Query Processing Architecture in MongoDB

Unlike SQL Server, which uses a cost-based optimizer dependent on complex statistics histograms to generate a query plan based on Relational Algebra, MongoDB employs a different approach tailored for document structures. The query processing lifecycle in MongoDB consists of three main stages: Parsing, Planning (Optimization), and Execution.

8.4.1 Phase 1: Parsing and Canonicalization

When a query (e.g., `find()` or `aggregate()`) is received, the parser checks for syntax errors and converts the query into an internal canonical form.

- For **Aggregation Framework** (used in Case Study 2 and 3): The optimizer attempts to reorganize the pipeline for efficiency. For example, if a `$match` stage follows a `$sort` stage, the optimizer may swap them to filter documents before sorting, reducing memory usage.

8.4.2 Phase 2: Query Planning and Optimization

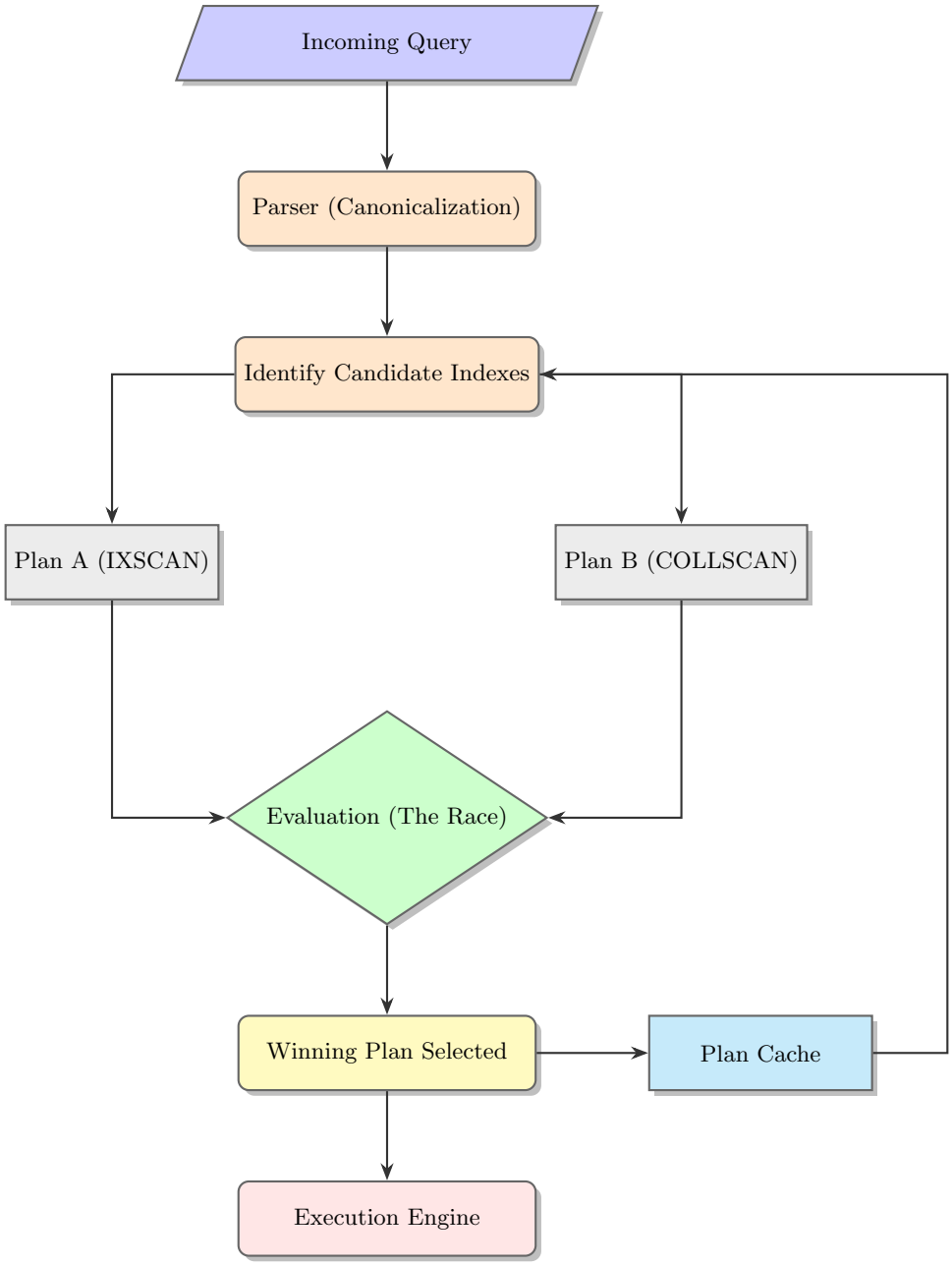


Figure 38: MongoDB Query Optimizer: The Empirical "Race" Model

This is the critical phase where MongoDB decides *how* to execute the query. Unlike SQL Server's strictly cost-based model, MongoDB's optimizer often uses an **empirical approach**:

1. **Plan Enumeration:** The optimizer identifies all possible candidate plans. For example, in Case Study 1 (Section 8.2), initially, the only plan was a collection scan (`COLLSCAN`). After indexing, two plans existed: scan the whole collection OR use the B-tree index (`fileName_1`).
2. **Plan Evaluation (The Race):** MongoDB may run the candidate plans in parallel for a trial period or a specific number of "work units." The plan that produces results most efficiently (fewest documents examined, lowest work units) is chosen as the **Winning Plan**.

3. **Plan Caching:** The winning plan is stored in the *Plan Cache*. Future queries with the same "query shape" (structure) will skip the evaluation phase and reuse the cached plan.

Referring to the JSON output in Case Study 1 (Figure **??**), the `"winningPlan"` object confirms that the optimizer explicitly chose the input stage `IXSCAN` over other possibilities.

8.4.3 Phase 3: Query Execution

The execution engine processes the documents according to the winning plan using a retrieval tree. The key operators observed in our case studies include:

- **COLLSCAN (Collection Scan):** The engine reads every document in the collection. This was observed in Section 8.2.1 where `totalDocsExamined` equaled the collection size (70,023), resulting in high latency (23ms).
- **IXSCAN (Index Scan):** The engine traverses the B-tree index. This is efficient ($O(\log n)$) and was triggered after creating the index on `fileName`, reducing execution time to 4ms.
- **FETCH:** This operator retrieves the full document from disk/memory based on the record ID found in the index. In Case Study 2, despite using an index, the `FETCH` stage was still necessary to get fields not in the index, creating a bottleneck.
- **PROJECTION (Covered Query):** As demonstrated in Case Study 3, when all requested fields are contained within the index, the `FETCH` stage is removed completely. The query is satisfied purely from the index keys, which is the most optimized state for read operations in MongoDB.

8.4.4 Comparison with SQL Server Processing

While both systems use B-trees and aim to minimize I/O:

- **SQL Server** focuses heavily on optimizing complex JOIN algorithms (Hash Match, Nested Loops) and uses algebraic restructuring.
- **MongoDB** focuses on optimizing the **Aggregation Pipeline** flow and selecting the best index to minimize document inspection (`totalDocsExamined`). The "Covering Index" strategy (removing Key Lookup/FETCH) is conceptually identical in both systems and yields the highest performance gains.

8.5 MongoDB Atomic Operations and Transactions

Unlike Microsoft SQL Server, which relies heavily on relational tables and transactions for consistency, MongoDB provides atomic operations at the document level and only requires multi-document transactions in specific scenarios.

8.5.1 Database Structure

Each MRI image is stored as a single document:

```
{
  "patientID": "...",
  "studyID": "...",
  "seriesID": "...",
  "imageFolder": "...",
  "fileName": "...",
  "path": "..."
}
```

This hierarchical, nested structure contrasts with SQL Server's normalized tables, where each piece of information would typically be stored in separate tables.

8.5.2 Atomic Insertion

Inserting a single MRI image document is inherently atomic:

```
doc = {
    "patientID": patient,
    "studyID": study,
    "seriesID": series,
    "imageFolder": image_folder,
    "fileName": image_folder,
    "path": img_folder_path.replace("\\", "/"),
    "version": 1,
    "verified": False,
    "annotations": []
}
col.insert_one(doc)
```

Implement 31: Atomic insertion in MongoDB

In SQL Server, this would require inserting into multiple tables if the data were normalized (patients, studies, series, images), and ensuring atomicity would require an explicit transaction.

8.5.3 Atomic Update of Multiple Fields

MongoDB allows updating multiple fields in a single document atomically:

```
col.update_one(
    {"_id": image_id},
    {
        "$set": {"path": "/new/path/IMG_001", "verified": True},
        "$inc": {"version": 1}
    }
)
```

Implement 32: Atomic update of multiple fields

In SQL Server, updating multiple related columns across tables would require a transaction. MongoDB does this automatically at the document level.

8.5.4 Atomic Update of Array Elements

Updating elements inside an array is also atomic:

```
col.update_one(
    {"_id": image_id},
    {"$push": {"annotations": {"author": "Dr. Smith", "coords": [120,88], "label": "lesion"}}}
)
```

Implement 33: Atomic array update

Relational databases cannot update a nested array directly. SQL Server would require a separate child table and a transaction to maintain consistency.

8.5.5 Upsert: Insert or Update Atomically

MongoDB can insert a new document if it does not exist, or update it if it does, in one atomic operation:

```
col.update_one(
    {
        "patientID": patient,
        "studyID": study,
        "seriesID": series,
```

```
        "fileName": image_folder
    },
    {
        "$set": {"path": img_folder_path.replace("\\", "/"), "lastImported": datetime.now()}
    },
    upsert=True
)
```

Implement 34: Atomic upsert

In SQL Server, this requires checking for existence and wrapping the logic in a transaction. MongoDB does it in a single command.

8.5.6 Atomic Counters

Counters such as total imported images per patient can be incremented atomically:

```
patients_col.update_one(
    {"patientID": patient},
    {"$inc": {"totalImages": 1}},
    upsert=True
)
```

Implement 35: Atomic counter increment

SQL Server would need a transaction with row locks to safely increment a counter concurrently.

8.5.7 Multi-Document Transaction

When multiple documents must be updated together, MongoDB provides multi-document transactions:

```
session = client.start_session()
try:
    session.start_transaction()
    col.update_many(
        {"seriesID": "Series_2"},
        {"$set": {"studyID": "Study_B"}},
        session=session
    )
    session.commit_transaction()
except PyMongoError as e:
    session.abort_transaction()
finally:
    session.end_session()
```

Implement 36: Transaction example

While SQL Server always uses transactions for multiple table updates, MongoDB only requires them for multi-document operations, not for single-document atomic updates.

8.5.8 Summary of Differences with SQL Server

- MongoDB guarantees **atomicity at the document level**, eliminating the need for transactions in many cases.
- SQL Server requires transactions whenever multiple tables or rows must be updated consistently.
- MongoDB can update arrays, embedded documents, and counters atomically within a single document.
- Multi-document transactions in MongoDB exist but are optional and come with a performance cost; SQL Server uses transactions by default for multi-row/table consistency.

8.6 MongoDB Concurrency Control with Write Conflicts

MongoDB uses **optimistic concurrency control** with **document-level locking**. In this example, we simulate two concurrent sessions attempting to update the `same document` to observe how conflicts are handled.

8.6.1 Python Implementation

```
import threading
import time
from pymongo import MongoClient
from pymongo.errors import PyMongoError
from datetime import datetime

def log(msg):
    print(f"[{datetime.now().strftime('%H:%M:%S')}] {msg}")

def update_session(session_name, delay=5):
    client = MongoClient("mongodb://localhost:27017/?replicaSet=rs0")
    db = client["mri_db"]
    coll = db["mri_images"]

    with client.start_session() as session:
        while True:  # retry loop
            try:
                with session.start_transaction():
                    log(f"{session_name} - Transaction Started.")

                    result = coll.update_one(
                        {"patientID": "01_MRI_Data"},
                        {"$set": {"description": f"Updated by {session_name}"}},
                        session=session
                    )
                    log(f"{session_name} - Update command sent. Matched={result.matched_count}")

                    if delay > 0:
                        log(f"{session_name} - Holding before commit for {delay}s...")
                        time.sleep(delay)

                    log(f"{session_name} - Committing transaction...")
                    session.commit_transaction()
                    log(f"{session_name} - Commit complete.")
                    break

            except PyMongoError as e:
                if hasattr(e, 'details') and e.details and 'errorLabels' in e.details:
                    if 'TransientTransactionError' in e.details['errorLabels']:
                        log(f"{session_name} - WriteConflict / TransientTransactionError detected, retrying...")
                        time.sleep(1)
                        continue
                log(f"{session_name} - ERROR: {e}")
                break

t1 = threading.Thread(target=update_session, args=("Session 1", 5))
t2 = threading.Thread(target=update_session, args=("Session 2", 5))

t1.start()
time.sleep(1)
t2.start()
```

```
t1.join()
t2.join()
```

Implement 37: MongoDB WriteConflict Simulation with Retry

8.6.2 Console Output

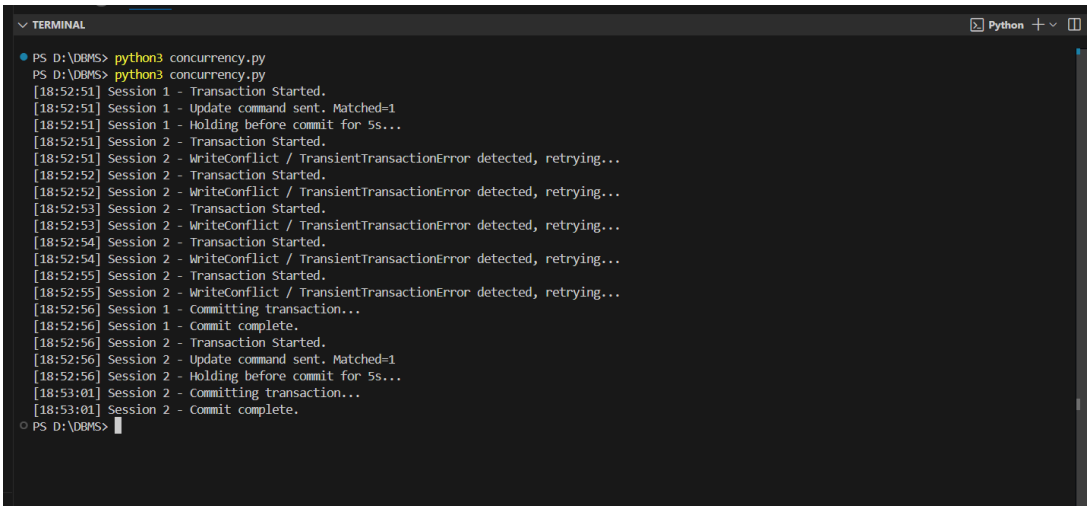


Figure 39: Log

8.6.3 Comparison with SQL Server

Feature	SQL Server (Relational)	MongoDB (Document)
Lock Granularity	Row, Page, or Table. Row locks can escalate to page/table locks, potentially blocking unrelated rows.	Document Level. Each JSON document is locked independently; other documents remain unaffected.
Default Behavior	Pessimistic. Writers can block readers and vice versa; snapshot isolation is optional.	Optimistic / MVCC. Writers do not block readers. Concurrent writes may conflict at commit time.
Blocking Scope	Potentially large if many rows are locked; depends on isolation level.	Minimal. Only conflicting transactions updating the same document may retry.
Conflict Handling	Deadlocks detected via wait-for graph; one session killed (Error 1205).	WriteConflict, TransientTransactionError triggers retry. No full table deadlock.

Table 18: Concurrency Control: SQL Server vs MongoDB

8.6.4 Conclusion

This experiment confirms that MongoDB employs optimistic concurrency control with document-level locks:

- Concurrent transactions attempting to update the same document may trigger a `WriteConflict / TransientTransactionError`.
- Retrying the transaction ensures eventual commit without blocking unrelated documents.
- MongoDB achieves high concurrency for workloads with random document updates.
- SQL Server, in contrast, uses pessimistic locks which may block other rows or escalate to page/table locks, ensuring serializability but potentially reducing concurrency.

9 Conclusion

This project explored the critical role of database internals and performance tuning. Through practical case studies on the LumbarMRI dataset, we demonstrated that efficient data retrieval relies heavily on understanding the underlying storage and processing mechanisms. Key takeaways include:

- **The Power of Indexing:** In both SQL Server and MongoDB, replacing full scans (`Table Scan`/`COLLSCAN`) with index seeks (`Index Seek`/`IXSCAN`) is the primary driver of performance. Advanced strategies like **Covering Indexes** further optimize performance by eliminating expensive lookups (`Key Lookup`/`FETCH`).
- **Workload-Specific Architectures:** We observed that while B-Trees are ideal for transactional queries (OLTP), **Columnstore Indexes** are essential for reducing I/O and CPU usage in analytical workloads (OLAP).
- **SQL Server vs. MongoDB:** While both systems share fundamental indexing principles, they diverge in query processing. SQL Server relies on rigorous cost-based algebraic optimization for complex joins, whereas MongoDB utilizes an empirical "race" model and aggregation pipelines suited for document structures.

Ultimately, there is no "one-size-fits-all" database. The choice between a Relational DBMS (SQL Server) and a Document Store (MongoDB) depends on the specific trade-offs between strict transactional consistency (ACID) and schema flexibility needed for the application.

10 Source Code

The complete source code for this project, including the Python ETL scripts: https://github.com/Thanhizme/Database-Management-Systems-CO3021-.git